



Image Processing

Lecture 5: Histogram Processing

By

Dr. Rasha Mahmoud Kamel

Image Histogram

- The histogram of an image is a graphical representation of the frequency of occurrence of each of the permitted pixel values in the image.

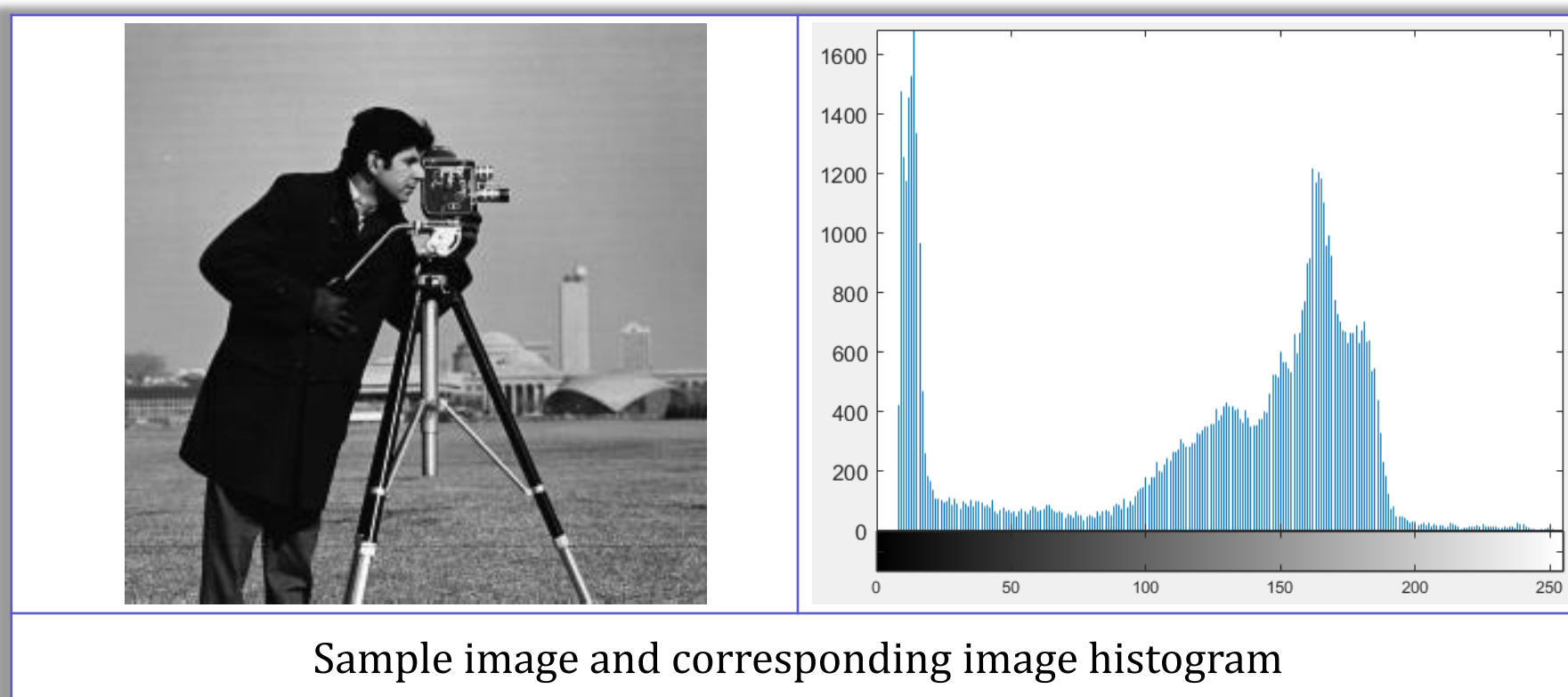


Image Histogram

- For a simple greyscale image, the histogram can be constructed by simply counting the number of times each greyscale value (0–255) occurs within the image.
- The unnormalized histogram of the image is defined as

$$h(r_k) = n_k \quad \text{for } k = 0, 1, \dots, L - 1$$

- where
 - L is the number of gray levels of the digitized image.
 - $r_k \in [0, L - 1]$ denotes the k th intensity value where $k = 0$ representing black and $k = L - 1$ representing white.
 - n_k is the number of pixels in the image with intensity r_k .

Image Histogram

- Therefore $h(r_0)$ is the number of pixels with intensity value r_0 , $h(r_1)$ the number of pixels with the value r_1 , and so forth.
- The result of the histogram computation is a one-dimensional vector h of length L whose individual elements store the number of image pixels that correspond to each gray level.

Image Histogram

- A normalized histogram can be mathematically defined as

$$p(r_k) = \frac{h(r_k)}{n} = \frac{n_k}{n}$$

- where

- $n = M \times N$ is the total number of pixels in the image. As usual, M and N are the number of image rows and columns, respectively.
- r_k for $k = 0, 1, \dots, L - 1$ denotes the k th intensity value.
- n_k is the number of pixels in the image with intensity r_k .
- $p(r_k)$ is the probability (percentage) of the k th gray level (r_k) occurring in an image.
- The sum of $p(r_k)$ for all values of k is always 1.

Image Histogram

- The following table shows the pixel counts and probabilities of each gray level in a 3-bit image ($L = 8$). This image is of size 128×128 pixels where the intensity levels are integers in the range $[0, L - 1] = [0, 7]$. The image's histogram is sketched in the following figure.

Gray Level (r_k)	n_k	$p(r_k)$
0	1120	0.068
1	3214	0.196
2	4850	0.296
3	3425	0.209
4	1995	0.122
5	784	0.048
6	541	0.033
7	455	0.028
Total	16,384	1.000

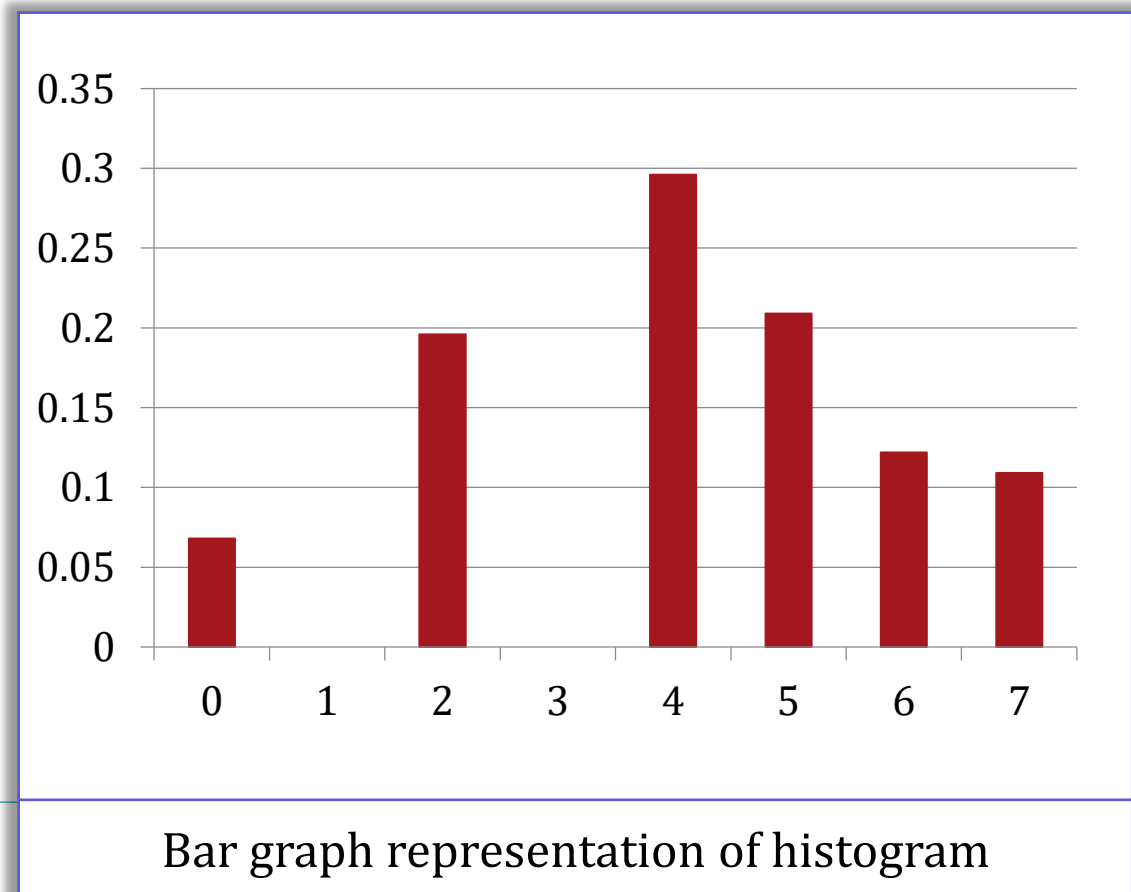


Image Histogram

- The number of pixels that correspond to a given gray level is indicated in the second column.
- The percentages (probabilities) of each gray level $p(r_k)$ are given in the third column.
- Each value of $p(r_k)$ represents the percentage of pixels in the image whose gray level is r_k .
- The sum of the values for $p(r_k)$ is 1.

Gray Level (r_k)	n_k	$p(r_k)$
0	1120	0.068
1	3214	0.196
2	4850	0.296
3	3425	0.209
4	1995	0.122
5	784	0.048
6	541	0.033
7	455	0.028
Total	16,384	1.000

Image Histogram

- **Example:** Given a 3-bit image ($L = 8$) of size 7×7 pixels where the intensity levels are integers in the range $[0, L - 1] = [0, 7]$. Calculate the probabilities of each gray level and plot the image's histogram.

0	3	2	2	1	1	4
3	2	1	1	1	1	4
2	0	7	1	3	0	0
0	5	1	0	6	0	1
1	1	1	3	6	6	5
1	1	1	1	5	6	4
0	7	0	0	0	0	4

Image Histogram

■ Solution:

Gray Level (r_k)	n_k	$p(r_k)$
0	12	0.245
1	18	0.367
2	4	0.082
3	4	0.082
4	4	0.082
5	3	0.061
6	4	0.082
7	2	0.041
Total	49	1.000

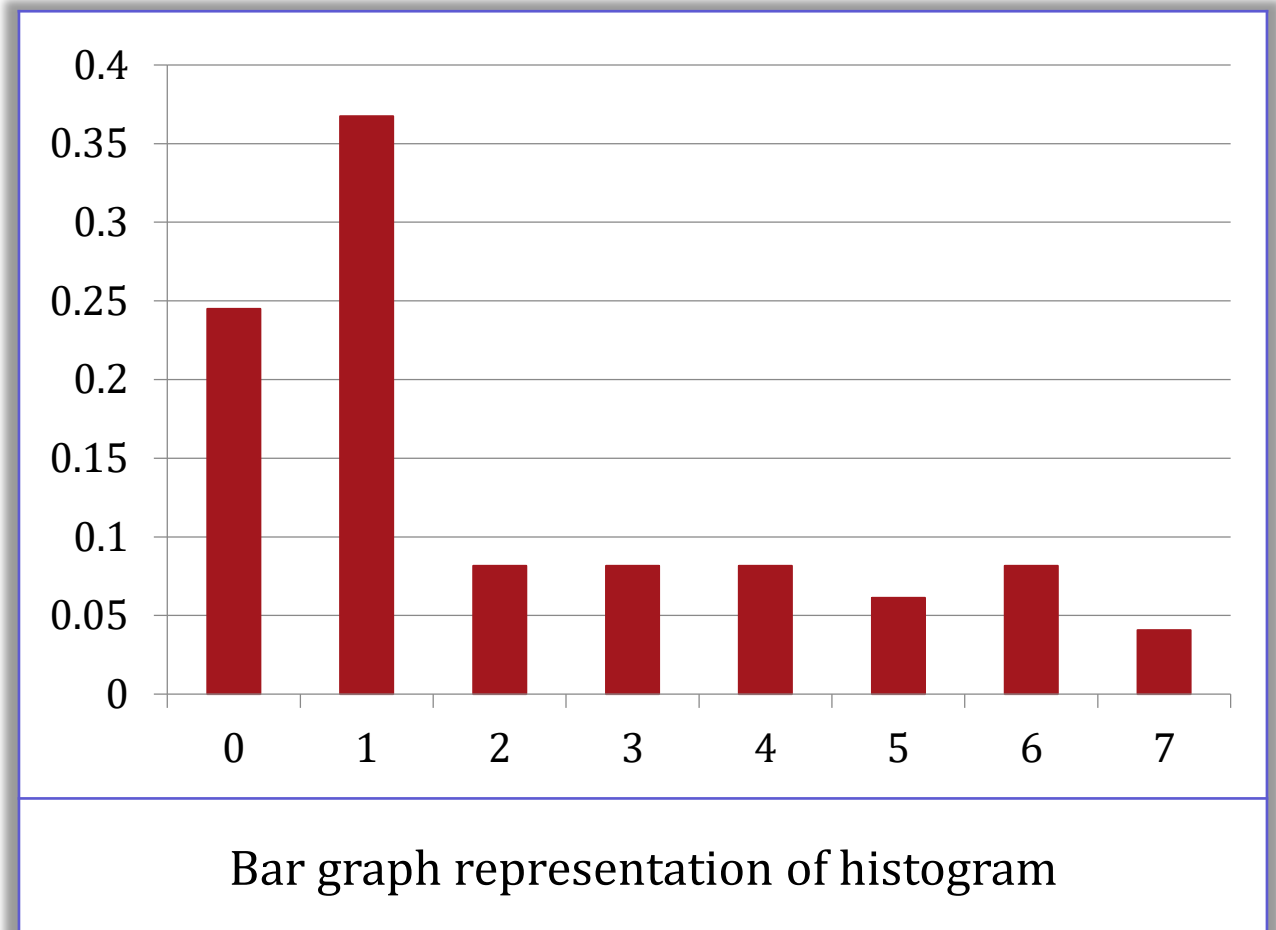


Image Histogram – MATLAB

- MATLAB's IPT has a built-in function called **imhist** that can be used to calculate and display the histogram of a monochrome image.

```
[counts, bins] = imhist ( Input_Img , nbins )
```

- **nbins** is a number of bins, specified as a positive integer, used to calculate the histogram. If `Img_input` is a grayscale image, then `imhist` uses a default value of 256 bins. If `Img_input` is a binary image, then `imhist` uses two bins.
- **counts** and **bins** are two numeric arrays of the same length.

Image Histogram – MATLAB

- **Example:** Open the circuit image in MATLAB. Display the image and its histogram.

```
Img_input = imread('circuit.tif');  
figure,  
subplot(1,2,1), imshow(Img_input),title('Image')  
subplot(1,2,2), imhist(Img_input), axis tight, title('Histogram')
```

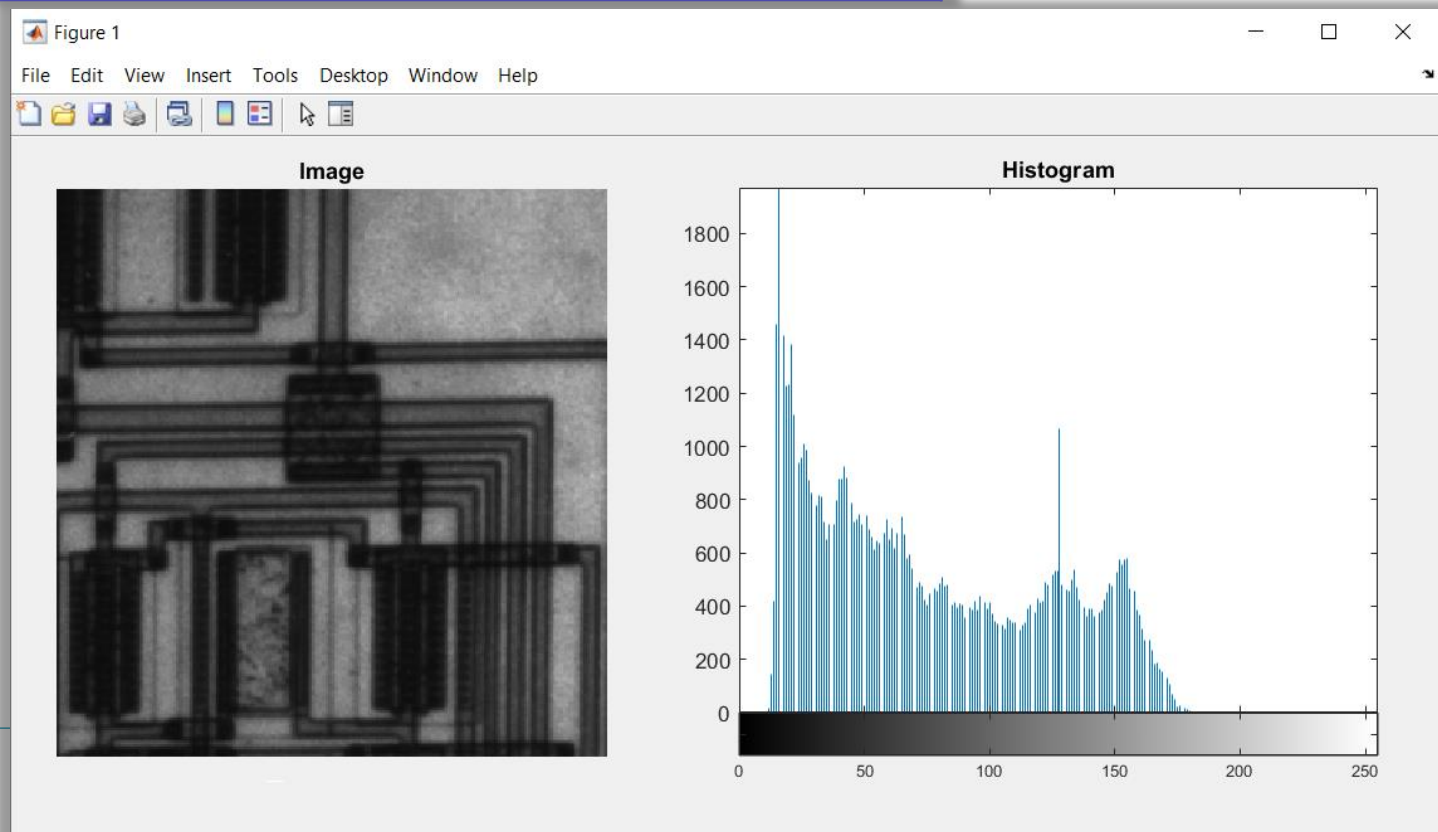


Image Histogram – MATLAB

- **Example:** Open the circuit image in MATLAB. Get the histogram bin values and the pixel count associated with each bin or a selected bin.

```
Img_input = imread('circuit.tif');  
[counts,bins] = imhist(Img_input,256);  
disp(counts(20))
```

counts		bins	
256x1 double		256x1 double	
	1		1
1	0	1	0
2	0	2	1
3	0	3	2
4	0	4	3
5	0	5	4
6	0	6	5
7	0	7	6
8	0	8	7
9	0	9	8
10	0	10	9
11	0	11	10
12	0	12	11
13	15	13	12
14	144	14	13
15	418	15	14
16	1459	16	15
17	1972	17	16
18	0	18	17
19	1416	19	18
20	1228	20	19
21	1232	21	20
22	1384	22	21
23	1119	23	22



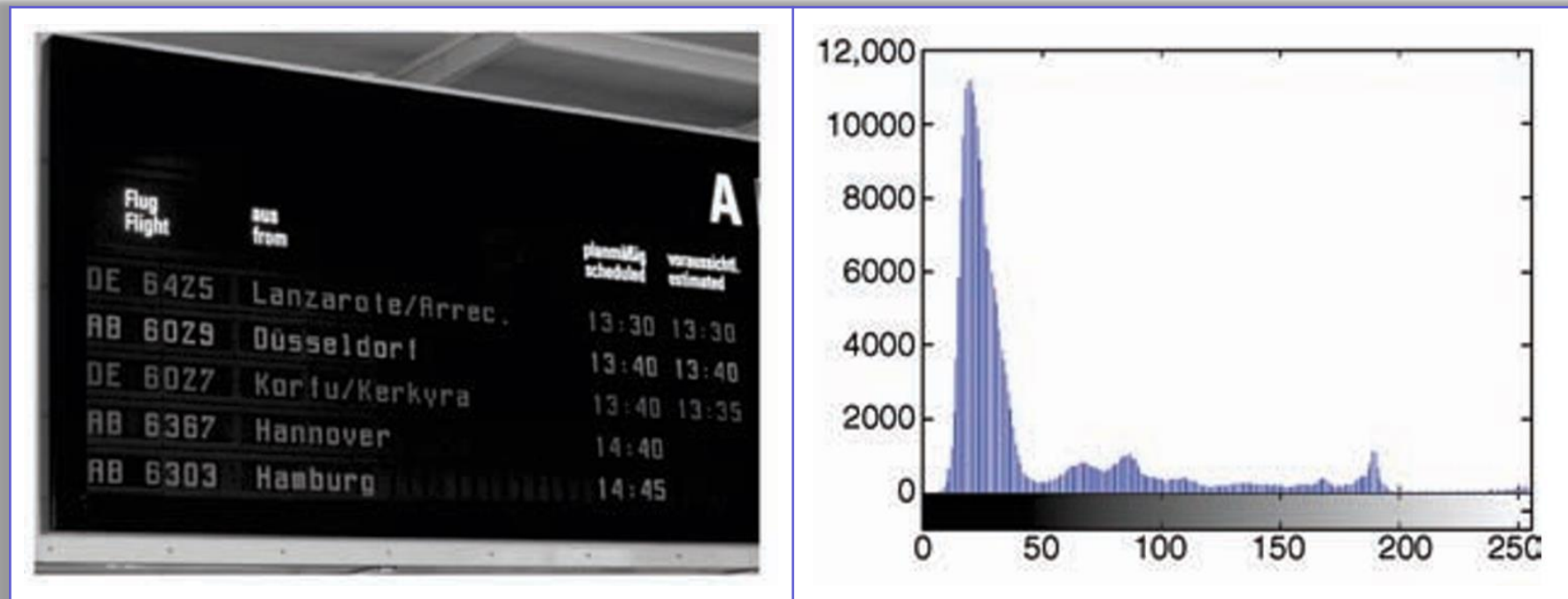
Computing Image Histograms

■ To compute the histogram of an 8-bit (256 gray levels) monochrome image:

- (1) An array of 256 elements (each of which acts as a counter) is created and initialized with zeros.
- (2) The image is then read, one pixel at a time, and for each pixel the array position corresponding to its gray level is incremented.
- (3) After the whole image is processed, each array element will contain the number of pixels whose gray level corresponds to the element's index.
- (4) These values can then be normalized, dividing each of them by the total number of pixels in the image.

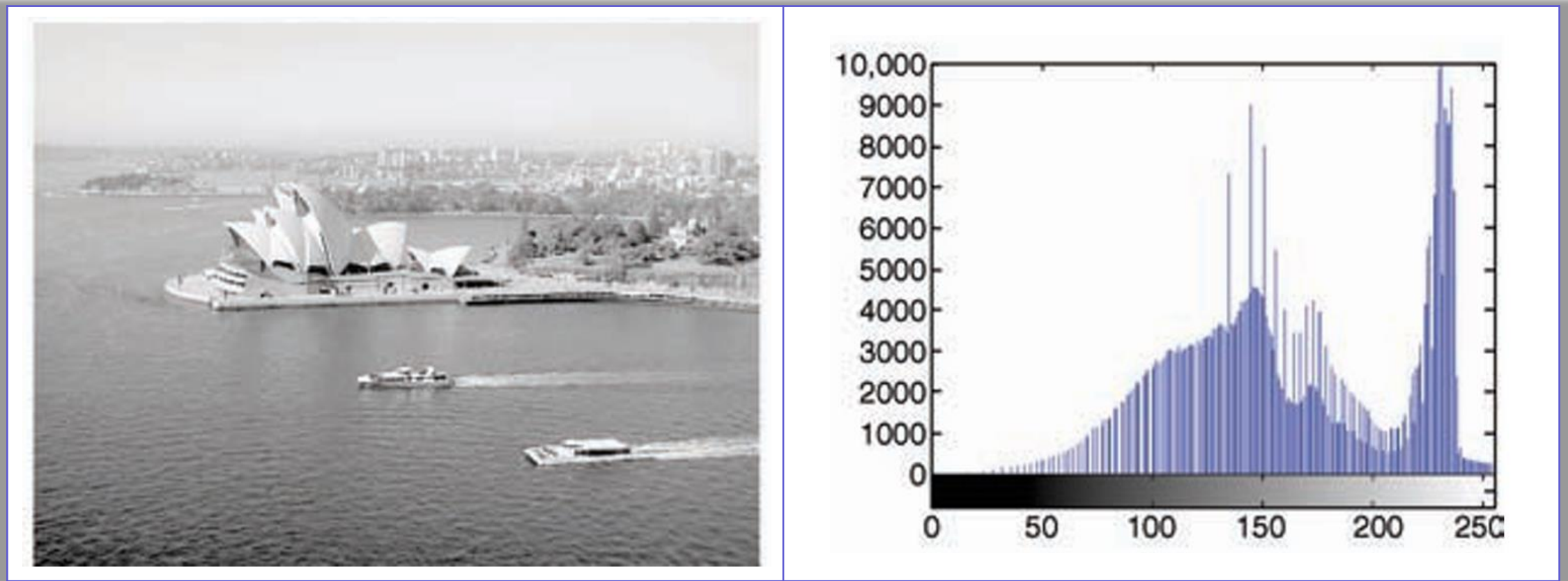
Interpreting Image Histograms

- Histograms provide an easy, practical, and straightforward way of evaluating image attributes, such as overall contrast and average brightness.
- The histogram of a predominantly dark image contains a concentration of bars on the lower end of the gray-level range (a large concentration of pixels in the lower gray levels).



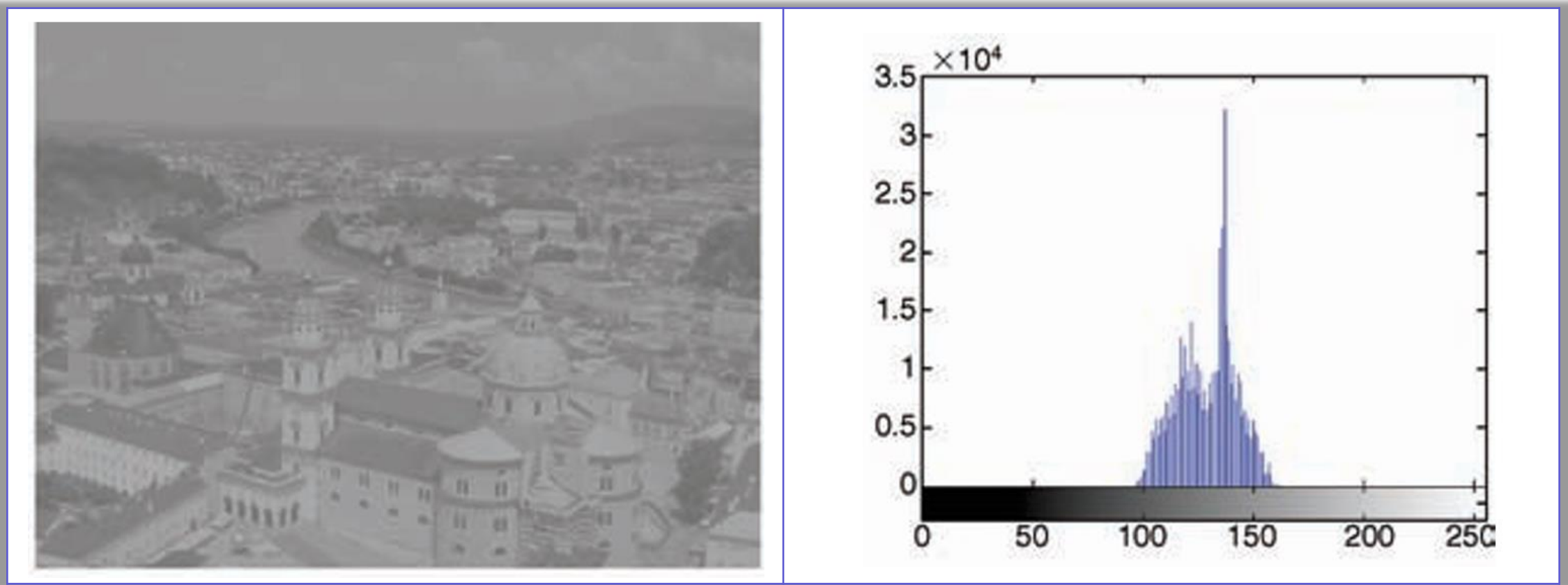
Interpreting Image Histograms

- The histogram for a bright image is mostly concentrated on the higher end of the gray-level range (the pixel values are grouped close to the higher gray-level values).



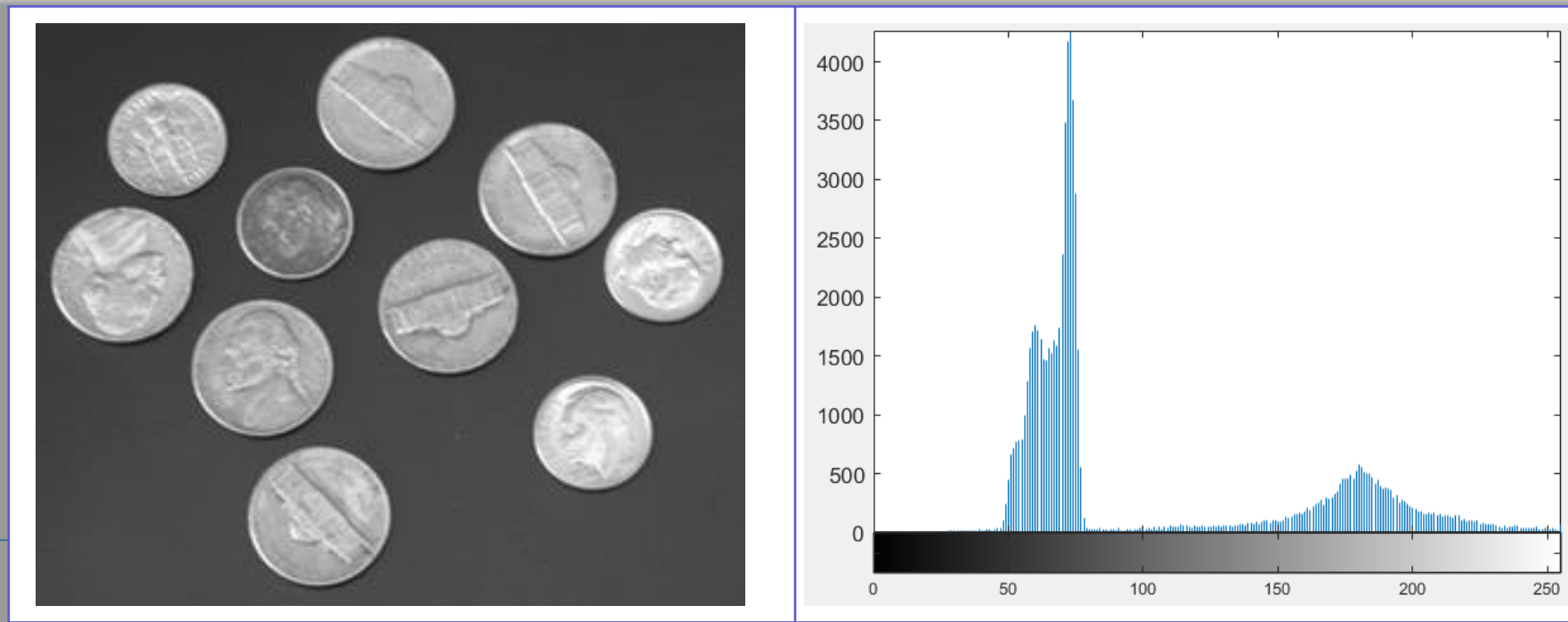
Interpreting Image Histograms

- The histogram for a low contrast image is clustered within a narrow range of gray levels. The pixels are grouped around intermediate gray-level values (mostly in the $[100, 150]$ range).



Interpreting Image Histograms

- A high contrast image usually exhibits a bimodal histogram with clear separation between the two predominant modes. A typical bimodal histogram is one that has two distinctive hills, a more pronounced one in the dark region (background) and the other smaller one in the light region of the histogram (foreground objects). In these situations, it can be said that the corresponding image has high contrast since the two modes are well spaced from each other.



Interpreting Image Histograms

- It is important to note that a histogram carries significant qualitative and quantitative information about the corresponding image (e.g., minimum, average, and maximum gray-level values, dominance of bright or dark pixels, etc.).
- However, other qualitative conclusions can be reached only upon examining the image itself (e.g., overall quality of the image, presence or absence of noise, etc.).
- Although a histogram provides the frequency distribution of gray levels in an image, it contains no information about the spatial arrangement of pixels in the image (where each of its individual entries originated in the image).

Interpreting Image Histograms

- Histograms have become a popular tool for conveying image statistics and helping determine certain problems in an image.
- Histograms can be used to enhance or modify the characteristics of an image, particularly its contrast.
- Some of these techniques, generally called histogram modification (or modeling) techniques, are histogram equalization, histogram specification (matching), histogram stretching (input cropping), and histogram shrinking (output cropping).

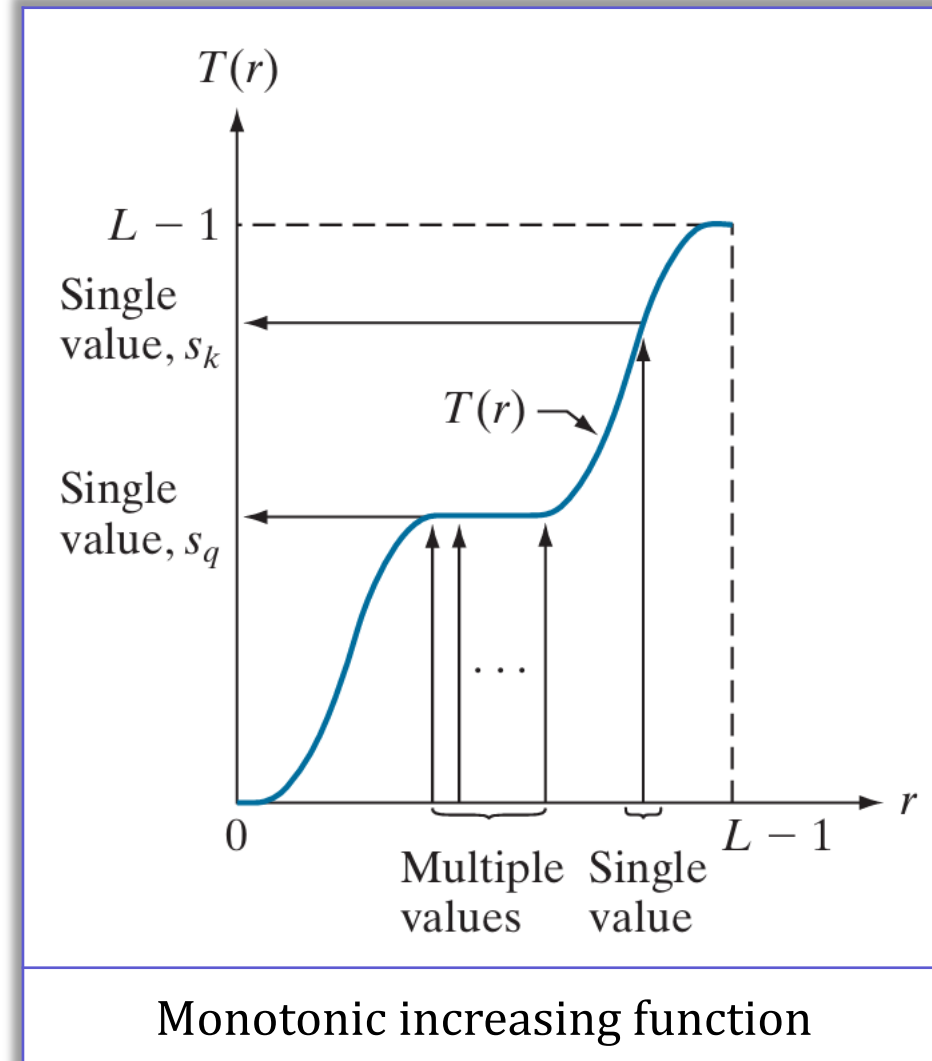
Histogram Equalization

- **Histogram Equalization** is an image processing technique used to improve contrast in images.
- It is a technique by which the gray-level distribution of an image is changed in such a way as to obtain a uniform (flat) resulting histogram, in which the percentage of pixels of every gray level is the same.
- The basic idea of the histogram equalization problem is that we seek a transformation function $T(r)$ that maps between the input and the output greyscale image values.

Histogram Equalization

■ Such transformation function must satisfy two criteria:

- (1) $T(r)$ is a monotonic increasing function in the interval $0 \leq r \leq L - 1$. This guarantees that output intensity values will never be less than corresponding input values.
- (2) $0 \leq T(r) \leq L - 1$ for $0 \leq r \leq L - 1$. This guarantees that the range of output intensities is the same as the input.



Histogram Equalization

- The most usual transformation function is the **cumulative distribution function** (cdf) of the original probability mass function, given by

$$s_k = T(r_k) = (L - 1) \sum_{j=0}^k \frac{n_j}{n} = (L - 1) \sum_{j=0}^k p(r_j) \quad \text{for } k = 0, 1, \dots, L - 1$$

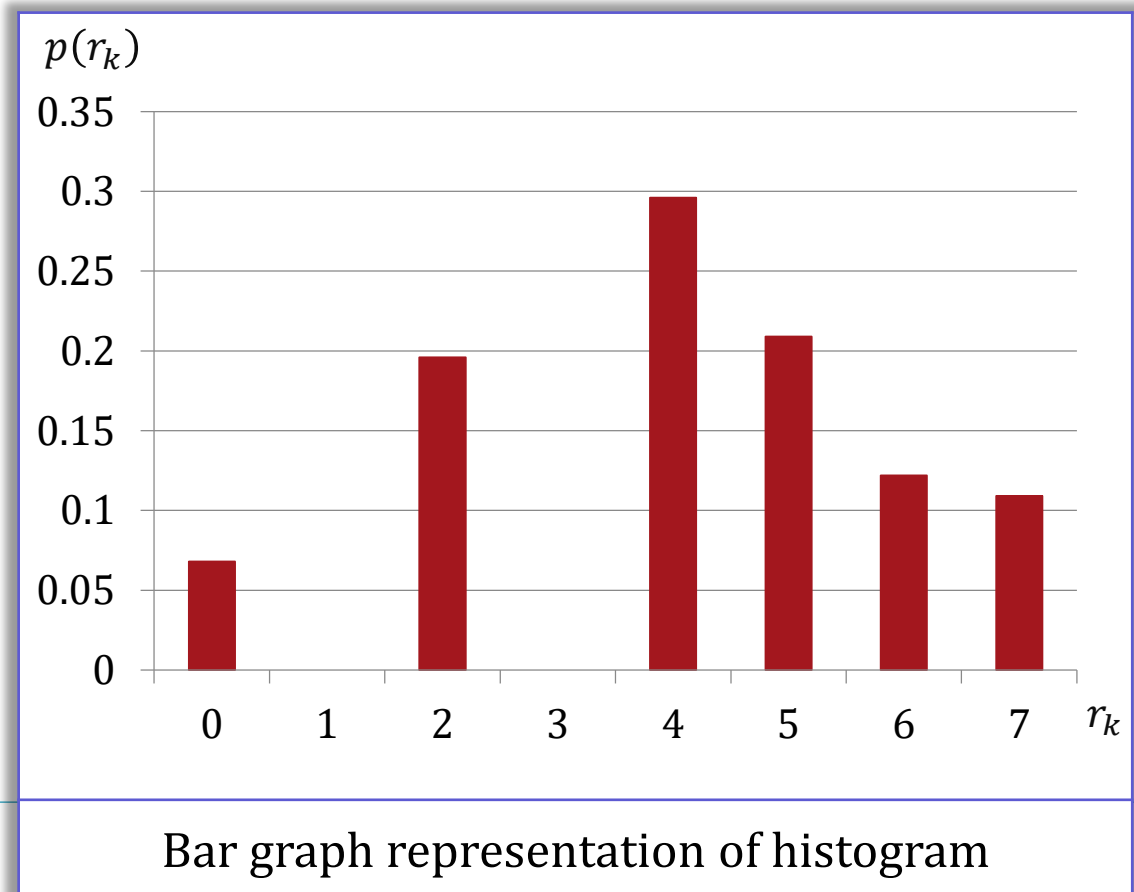
where s_k is the new (mapped) gray level for all pixels whose original gray level used to be r_k .

- Thus, a processed (output) image is obtained by using the above equation to map each pixel in the input image with intensity r_k into a corresponding pixel with level s_k in the output image, This is called a **histogram equalization** or **histogram linearization transformation**.

Histogram Equalization

- **Example:** Given the histogram data for a 3-bit image of size 128×128 pixels with eight gray levels. Calculate the equalized histogram using the cdf of the original probability mass function as a transformation function and plot the resulting histogram.

Gray Level (r_k)	n_k	$p(r_k)$
0	1120	0.068
1	3214	0.196
2	4850	0.296
3	3425	0.209
4	1995	0.122
5	784	0.048
6	541	0.033
7	455	0.028
Total	16,384	1.000



Histogram Equalization

■ Solution:

(1) Using the cdf as the transformation function, we can calculate

$$s_0 = T(r_0) = 7 \sum_{j=0}^k p(r_j) = p(r_0) = 0.476$$

$$s_1 = T(r_1) = 7 \sum_{j=0}^k p(r_j) = p(r_0) + p(r_1) = 1.848$$

$$s_2 = T(r_2) = 7 \sum_{j=0}^k p(r_j) = p(r_0) + p(r_1) + p(r_2) = 3.920$$

$$s_3 = T(r_3) = 7 \sum_{j=0}^k p(r_j) = p(r_0) + p(r_1) + p(r_2) + p(r_3) = 5.383$$

$$s_4 = T(r_4) = 7 \sum_{j=0}^k p(r_j) = 6.237$$

$$s_5 = T(r_5) = 7 \sum_{j=0}^k p(r_j) = 6.573$$

$$s_6 = T(r_6) = 7 \sum_{j=0}^k p(r_j) = 6.804$$

$$s_7 = T(r_7) = 7 \sum_{j=0}^k p(r_j) = 7.00$$

Histogram Equalization

■ Solution:

(2) At this point, the s values are fractional because they were generated by summing probability values, so we round them to their nearest integer values in the range $[0,7]$:

$s_0 = T(r_0) = 0.476 \rightarrow 0$	$s_4 = T(r_4) = 6.237 \rightarrow 6$
$s_1 = T(r_1) = 1.848 \rightarrow 2$	$s_5 = T(r_5) = 6.573 \rightarrow 7$
$s_2 = T(r_2) = 3.920 \rightarrow 4$	$s_6 = T(r_6) = 6.804 \rightarrow 7$
$s_3 = T(r_3) = 5.383 \rightarrow 5$	$s_7 = T(r_7) = 7.00 \rightarrow 7$

Rounded values of the transformation function $T(r_k)$

r_k	$T(r_k)$
$r_0 = 0$	0
$r_1 = 1$	2
$r_2 = 2$	4
$r_3 = 3$	5
$r_4 = 4$	6
$r_5 = 5$	7
$r_6 = 6$	7
$r_7 = 7$	7

Histogram Equalization

■ Solution:

(3) The histogram bars must be shifted and grouped according to the following mapping:

The 1120 pixels whose original gray level was $r_0 = 0$ should be mapped to the new level $s_0 = 0$, which means the corresponding bar should not change.

The 3214 pixels whose gray level was $r_1 = 1$ should be mapped to $s_1 = 2$.

The 4850 pixels whose gray level was $r_2 = 2$ should be mapped to $s_2 = 4$.

The 3425 pixels whose gray level was $r_3 = 3$ should be mapped to $s_3 = 5$.

The 1995 pixels whose gray level was $r_4 = 4$ should be mapped to $s_4 = 6$.

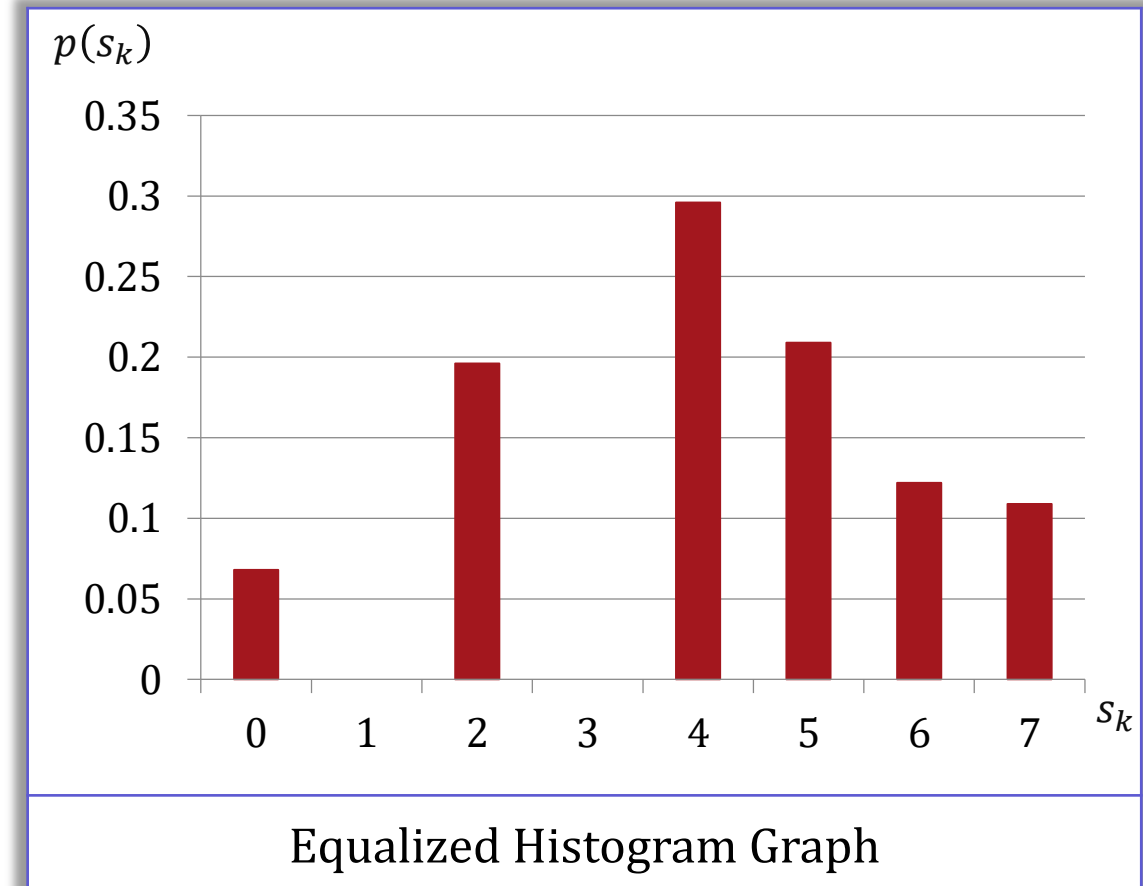
The three bins that correspond to pixels with gray levels 5, 6, and 7 should be added and mapped to 7.

Histogram Equalization

■ Solution:

Equalized Histogram

Gray Level (s_k)	n_k	$p(s_k)$
0	1120	0.068
1	0	0.000
2	3214	0.196
3	0	0.000
4	4850	0.296
5	3425	0.209
6	1995	0.122
7	1780	0.109
Total	16,384	1.000



Histogram Equalization – MATLAB

- MATLAB's IPT has a built-in function called **histeq** that can be used to perform histogram equalization of a monochrome image.

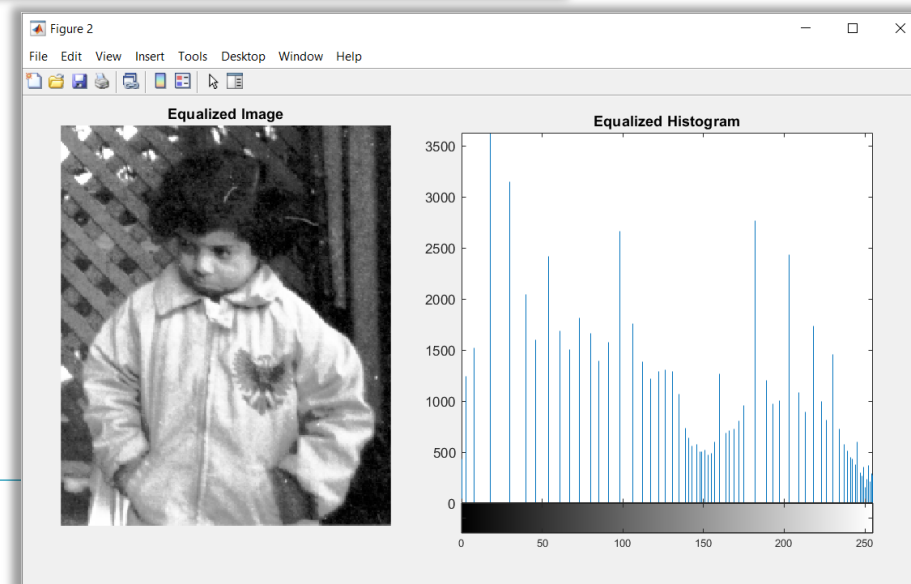
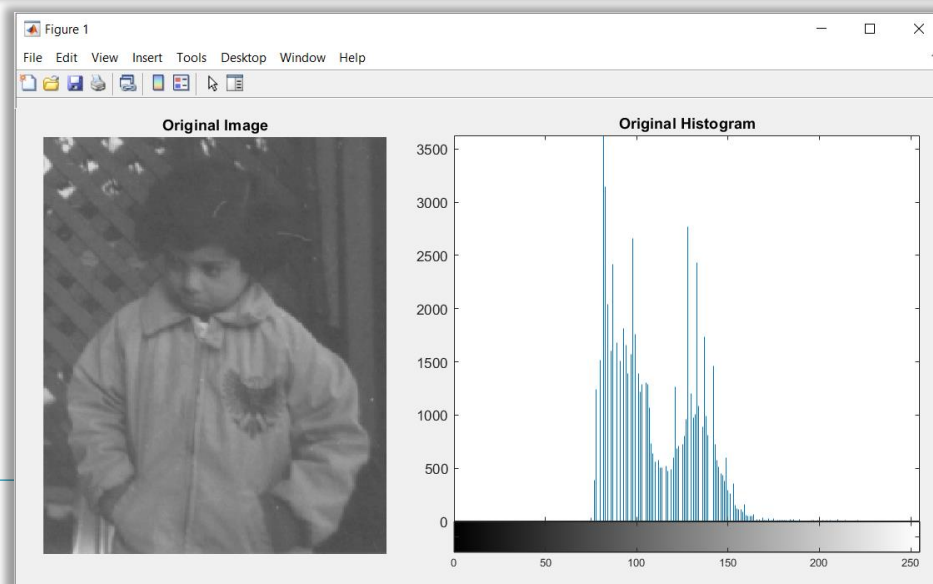
```
Output_Img = histeq ( Input_Img , nbins )
```

where **nbins** (whose default value is 64) is the number of desired gray levels in the output image.

Histogram Equalization – MATLAB

- **Example:** Perform histogram equalization on the pout image in MATLAB. Display the image and its histogram before and after performing histogram equalization.

```
Img_input = imread('pout.tif');  
Img_output = histeq(Img_input,256);  
figure  
subplot(1,2,1), imshow(Img_input), title('Original Image')  
subplot(1,2,2), imhist(Img_input), axis tight, title('Original Histogram')  
figure  
subplot(1,2,1), imshow(Img_output), title('Equalized Image')  
subplot(1,2,2), imhist(Img_output), axis tight, title('Equalized Histogram')
```



Adaptive Histogram Equalization

- The histogram equalization algorithm is global. That is, once the mapping (transformation) function is computed, it is applied to all pixels of the input image.
- To enhance details in small areas within an image, a local version of the algorithm is applied.
- The local variant of histogram equalization consists of adopting a rectangular (usually square) sliding window (also called a tile) and moving it across the entire image.
- For each image pixel (aligned with the center of the sliding window), the histogram in the neighborhood delimited by the window is computed, the mapping function is calculated, and the reference pixel is remapped to the new value as determined by the transformation function.
- This process is clearly much more computationally expensive than its global variant.

Adaptive Histogram Equalization – MATLAB

- MATLAB's IPT has also a built-in function called **adapthisteq** that can be used to perform local histogram equalization of a monochrome image.
- This function performs contrast limited adaptive histogram equalization (CLAHE). Unlike histeq, it operates on small data regions (called tiles), whose size can be passed as a parameter.

```
Output_Img = adapthisteq (Input_Img , Name, Value )
```

where **Name** (e.g., **Nbins**, **NumTiles**, etc) is the argument name that must appear inside quotes and **Value** is the corresponding value.

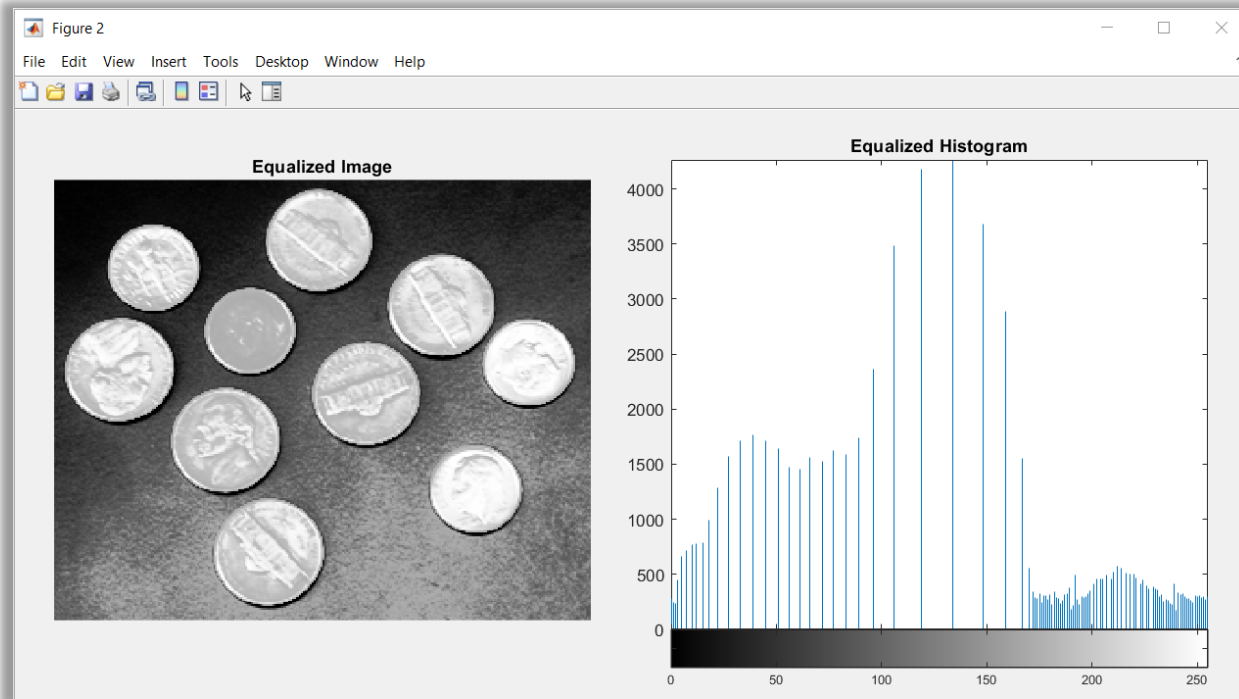
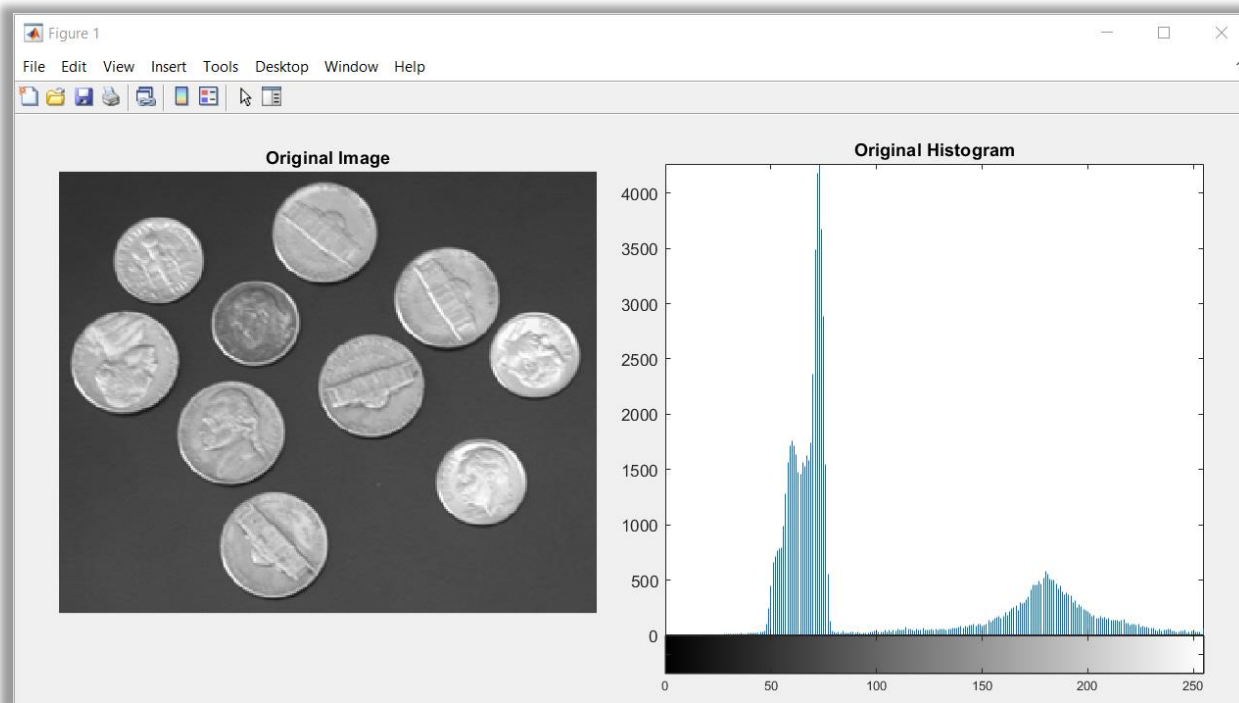
Adaptive Histogram Equalization – MATLAB

- **NBins** is a positive integer (whose default value is 256) which specifies the number of histogram bins used to build a contrast enhancing transformation.
- **NumTiles** is a 2-element vector of positive integers $[M \ N]$ which specifies the number of rectangular contextual regions (tiles) into which `adapthisteq` divides the image, Both M and N must be at least 2. The original image divided into M rows and N columns of tiles. The total number of tiles is equal to $M*N$.

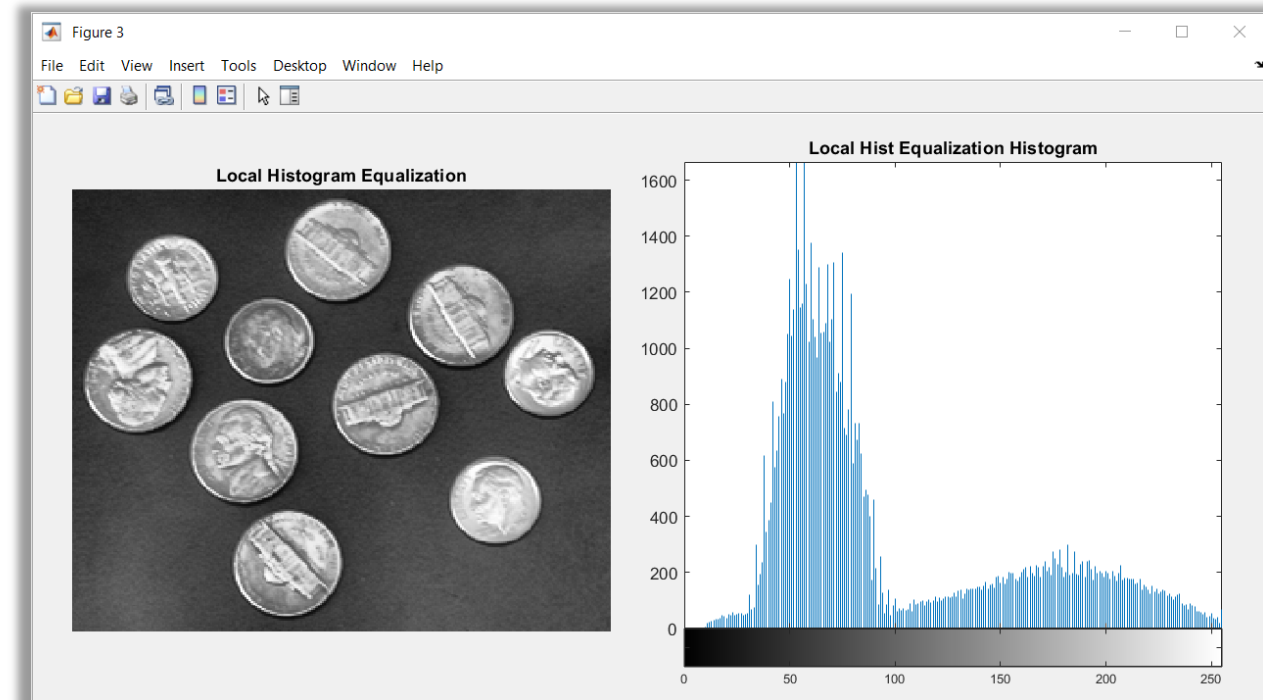
Adaptive Histogram Equalization – MATLAB

- **Example:** Perform local histogram equalization on the coins image in MATLAB. Display the image and its histogram before and after performing local histogram equalization.

```
Img_input = imread('coins.png');
Img_Eq = histeq(Img_input,256);
Img_LEq = adapthisteq(Img_input,'NumTiles',[4 4]);
figure,
subplot(1,2,1), imshow(Img_input), title('Original Image')
subplot(1,2,2), imhist(Img_input), axis tight, title('Original Histogram')
figure,
subplot(1,2,1), imshow(Img_Eq), title('Equalized Image')
subplot(1,2,2), imhist(Img_Eq), axis tight, title('Equalized Histogram')
figure,
subplot(1,2,1), imshow(Img_LEq),title('Local Histogram Equalization')
subplot(1,2,2), imhist(Img_LEq), axis tight, title('Local Hist EqualizationHistogram')
```



Local histogram equalization preserves the bimodal nature of the original histogram while still improving the contrast of the image.



Histogram Matching (Specification)

- Histogram equalization produces a transformation function that seeks to generate an output image with a uniform histogram.
- Despite the usefulness of histogram equalization in contrast enhancement. There are many situations in which equalization produces quite undesirable effects.
- In particular, it is useful sometimes to be able to specify the shape of the histogram that we wish the processed image to have.
- The method used to generate images that have a specified histogram is called **histogram matching** or **histogram specification**.

Histogram Matching (Specification)

- The basic idea of histogram matching is very simple.
- Given an input image (with its corresponding histogram) and the desired histogram, we seek to effect a transformation on the intensity values of the input image such that the transformed (output) image has a desired (target) histogram.
- Often, the target histogram will be taken from a model or ‘typical’ image of a similar kind.
- The procedure for histogram specification can be summarized as follows:
 - (1) Compute the normalized histogram of the input image

$$p(r_k) = \frac{h(r_k)}{n} = \frac{n_k}{n}$$

Histogram Matching (Specification)

- The procedure for histogram specification can be summarized as follows:

(1) Compute the normalized histogram of the input image

$$p(r_k) = \frac{h(r_k)}{n} = \frac{n_k}{n}$$

(2) Equalize the original image's histogram using the cdf as a transformation function:

$$s_k = T(r_k) = (L - 1) \sum_{j=0}^k \frac{n_j}{n} = (L - 1) \sum_{j=0}^k p(r_j) \quad \text{for } k = 0, 1, \dots, L - 1$$

(3) Round the resulting values of s_k to integers in the range $[0, L - 1]$.

Histogram Matching (Specification)

- (4) Form the equalized histogram by mapping every pixel whose original gray level is r_k to the new level s_k using the mappings found in Step 2-3.
- (5) Equalize the desired histogram (the desired probability mass function)

$$v_q = G(z_q) = (L - 1) \sum_{j=0}^q p(z_j) \quad \text{for } q = 0, 1, \dots, L - 1$$

where $p(z_q)$ are the values of the specified histogram.

- (6) Round the resulting values of v_q to integers in the range $[0, L - 1]$.

Histogram Matching (Specification)

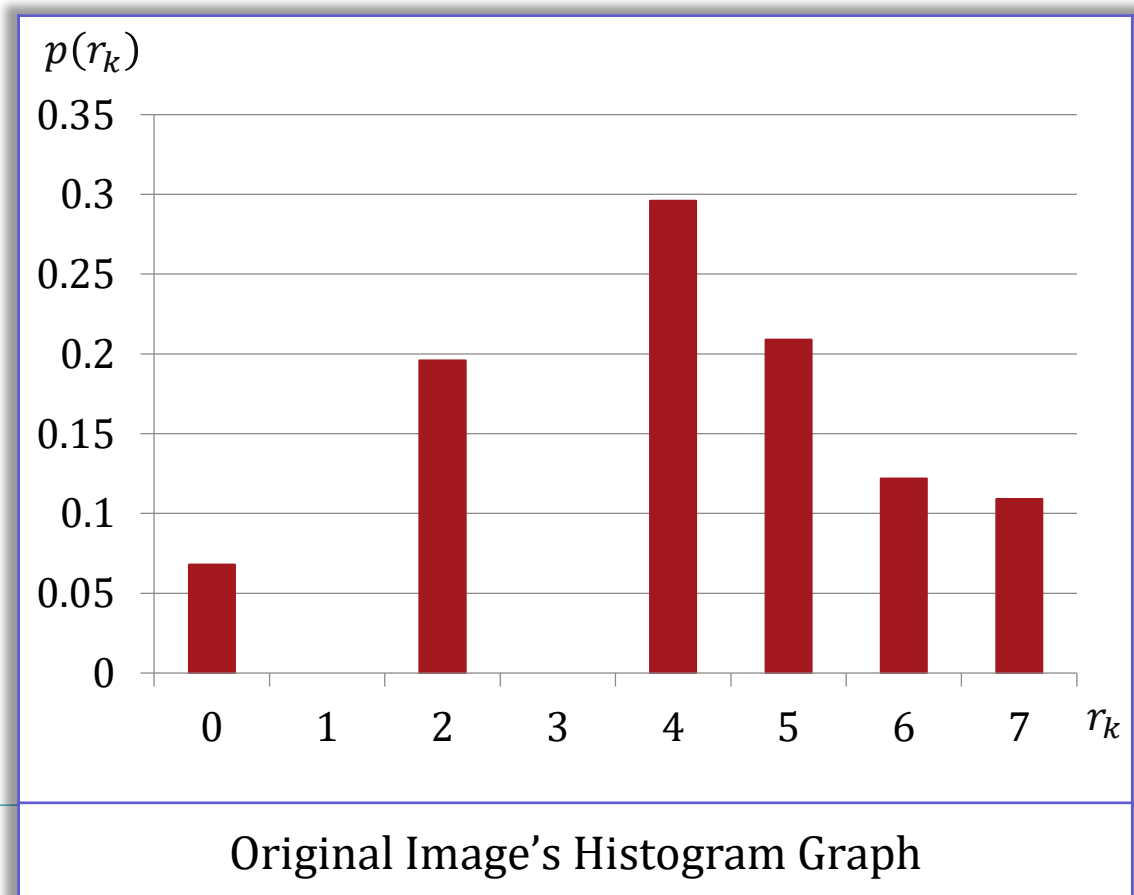
- (7) For every value of $s_k, k = 0, 1, \dots, L - 1$, use the values of v_q from Step 5 to find the corresponding value of z_q so that v_q is closest to s_k . Store these mappings from s to z . When more than one value of z_q gives the same match (i.e., the mapping is not unique), choose the smallest value by convention.
- (8) Form the histogram-specified image by mapping every equalized pixel with value s_k to the corresponding pixel with value z_q in the histogram-specified image using the mappings found in Step 7.

Histogram Matching (Specification)

- **Example:** Given the histogram data for a 3-bit image of size 128×128 pixels with eight gray levels. Assume that we want to modify the original image's histogram in such a way as to have

Original Image's Histogram

Gray Level (r_k)	n_k	$p(r_k)$
0	1120	0.068
1	3214	0.196
2	4850	0.296
3	3425	0.209
4	1995	0.122
5	784	0.048
6	541	0.033
7	455	0.028
Total	16,384	1.000

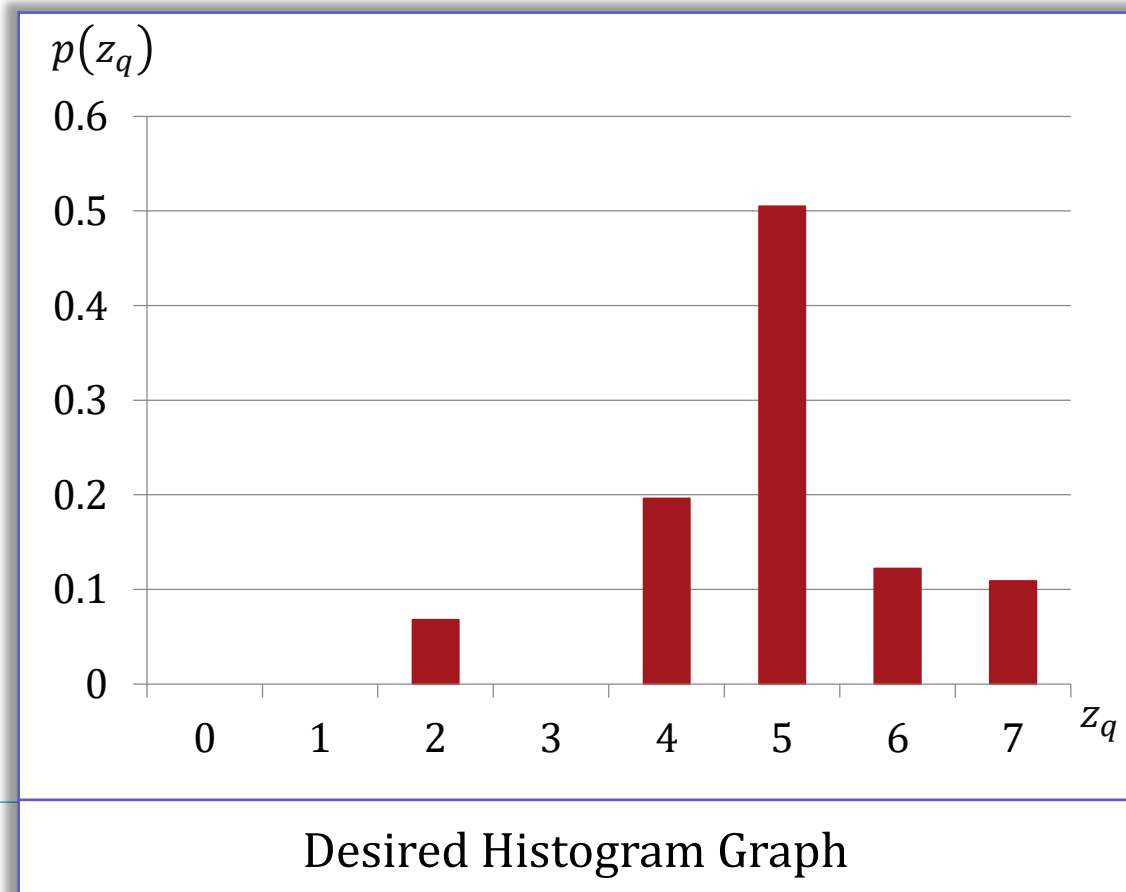


Histogram Matching (Specification)

the resulting pixel distribution (the transformed image has a desired target histogram) as shown below. Calculate and plot the resulting histogram that best matches the desired characteristics.

Desired Histogram

Gray Level (z_q)	n_q	$p(z_q)$
0	0	0.00
1	0	0.00
2	0	0.00
3	1638	0.1
4	3277	0.2
5	6554	0.4
6	3277	0.2
7	1638	0.1
Total	16,384	1.000



Histogram Matching (Specification)

■ Solution:

(2) Equalize the original image's histogram using the cdf transformation function:

$$s_0 = T(r_0) = 7 \sum_{j=0}^k p(r_j) = p(r_0) = 0.476$$

$$s_1 = T(r_1) = 7 \sum_{j=0}^k p(r_j) = p(r_0) + p(r_1) = 1.848$$

$$s_2 = T(r_2) = 7 \sum_{j=0}^k p(r_j) = p(r_0) + p(r_1) + p(r_2) = 3.920$$

$$s_3 = T(r_3) = 7 \sum_{j=0}^k p(r_j) = p(r_0) + p(r_1) + p(r_2) + p(r_3) = 5.383$$

$$s_4 = T(r_4) = 7 \sum_{j=0}^k p(r_j) = 6.237$$

$$s_5 = T(r_5) = 7 \sum_{j=0}^k p(r_j) = 6.573$$

$$s_6 = T(r_6) = 7 \sum_{j=0}^k p(r_j) = 6.804$$

$$s_7 = T(r_7) = 7 \sum_{j=0}^k p(r_j) = 7.00$$

Histogram Matching (Specification)

■ Solution:

(3) Round the resulting values of s_k to integers in the range $[0, 7]$:

$s_0 = T(r_0) = 0.476 \rightarrow 0$	$s_4 = T(r_4) = 6.237 \rightarrow 6$
$s_1 = T(r_1) = 1.848 \rightarrow 2$	$s_5 = T(r_5) = 6.573 \rightarrow 7$
$s_2 = T(r_2) = 3.920 \rightarrow 4$	$s_6 = T(r_6) = 6.804 \rightarrow 7$
$s_3 = T(r_3) = 5.383 \rightarrow 5$	$s_7 = T(r_7) = 7.00 \rightarrow 7$

Rounded values of the transformation function $T(r_k)$

r_k	$T(r_k)$
$r_0 = 0$	0
$r_1 = 1$	2
$r_2 = 2$	4
$r_3 = 3$	5
$r_4 = 4$	6
$r_5 = 5$	7
$r_6 = 6$	7
$r_7 = 7$	7

Histogram Matching (Specification)

■ Solution:

(4) Form the equalized histogram by mapping every pixel whose original gray level is r_k to the new level s_k according to the following:

The 1120 pixels whose gray level was $r_0 = 0$ should be mapped to the new level $s_0 = 0$.

The 3214 pixels whose gray level was $r_1 = 1$ should be mapped to $s_1 = 2$.

The 4850 pixels whose gray level was $r_2 = 2$ should be mapped to $s_2 = 4$.

The 3425 pixels whose gray level was $r_3 = 3$ should be mapped to $s_3 = 5$.

The 1995 pixels whose gray level was $r_4 = 4$ should be mapped to $s_4 = 6$.

The three bins that correspond to pixels with gray levels 5, 6, and 7 should be added and mapped to 7.

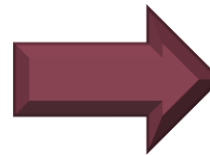
Histogram Matching (Specification)

■ Solution:

(4)

Original Image's Histogram

Gray Level (r_k)	n_k	$p(r_k)$
0	1120	0.068
1	3214	0.196
2	4850	0.296
3	3425	0.209
4	1995	0.122
5	784	0.048
6	541	0.033
7	455	0.028
Total	16,384	1.000



Equalized Histogram

Gray Level (s_k)	n_k	$p(s_k)$
0	1120	0.068
1	0	0.000
2	3214	0.196
3	0	0.000
4	4850	0.296
5	3425	0.209
6	1995	0.122
7	1780	0.109
Total	16,384	1.000

Histogram Matching (Specification)

■ Solution:

(5) Equalize the desired histogram (the desired probability mass function)

$$v_0 = G(z_0) = 7 \sum_{j=0}^k p(z_j) = p(z_0) = 0.00$$

$$v_1 = G(z_1) = 7 \sum_{j=0}^k p(z_j) = p(z_0) + p(z_1) = 0.00$$

$$v_2 = G(z_2) = 7 \sum_{j=0}^k p(z_j) = p(z_0) + p(z_1) + p(z_2) = 0.00$$

$$v_3 = G(z_3) = 7 \sum_{j=0}^k p(z_j) = p(z_0) + p(z_1) + p(z_2) + p(z_3) = 0.7$$

$$v_4 = G(z_4) = 7 \sum_{j=0}^k p(z_j) = 2.1$$

$$v_5 = G(z_5) = 7 \sum_{j=0}^k p(z_j) = 4.9$$

$$v_6 = G(z_6) = 7 \sum_{j=0}^k p(z_j) = 6.3$$

$$v_7 = G(z_7) = 7 \sum_{j=0}^k p(z_j) = 7.00$$

Histogram Matching (Specification)

■ Solution:

(6) Round the resulting values of v_k to integers in the range $[0, 7]$:

$v_0 = G(z_0) = 0.00 \rightarrow 0$	$v_4 = G(z_4) = 2.45 \rightarrow 2$
$v_1 = G(z_1) = 0.00 \rightarrow 0$	$v_5 = G(z_5) = 4.55 \rightarrow 5$
$v_2 = G(z_2) = 0.00 \rightarrow 0$	$v_6 = G(z_6) = 5.95 \rightarrow 6$
$v_3 = G(z_3) = 1.05 \rightarrow 1$	$v_7 = G(z_7) = 7.00 \rightarrow 7$

Rounded values of the transformation function $G(z_q)$

z_q	$G(z_q)$
$z_0 = 0$	0
$z_1 = 1$	0
$z_2 = 2$	0
$z_3 = 3$	1
$z_4 = 4$	2
$z_5 = 5$	5
$z_6 = 6$	6
$z_7 = 7$	7

Histogram Matching (Specification)

■ Solution:

(7) For every value of $s_k, k = 0, 1, \dots, L - 1$, find the closest value of v_q . That is, find the smallest value of z_q so that the value $v_q = G(z_q)$ is the closest to s_k . We do this for every value of s_k to create the required mappings from s to z .

$s_0 = 0 \rightarrow v_2 = G(z_2) = 0 \quad (s_0 \rightarrow z_2)$	$s_4 = 6 \rightarrow v_6 = G(z_6) = 6 \quad (s_4 \rightarrow z_6)$
$s_1 = 2 \rightarrow v_4 = G(z_4) = 2 \quad (s_1 \rightarrow z_4)$	$s_5 = 7 \rightarrow v_7 = G(z_7) = 7 \quad (s_5 \rightarrow z_7)$
$s_2 = 4 \rightarrow v_5 = G(z_5) = 5 \quad (s_2 \rightarrow z_5)$	$s_6 = 7 \rightarrow v_7 = G(z_7) = 7 \quad (s_6 \rightarrow z_7)$
$s_3 = 5 \rightarrow v_5 = G(z_5) = 5 \quad (s_3 \rightarrow z_5)$	$s_7 = 7 \rightarrow v_7 = G(z_7) = 7 \quad (s_7 \rightarrow z_7)$

Histogram Matching (Specification)

■ Solution:

(7) Note that, for a given value of s_k , we search through the values of v_k and store the index of the most recent (i.e., last) value.

Histogram Specification: Summary

$k = 0$	s_k	Maps to
0	0	z_2
1	2	z_4
2	4	z_5
3	5	z_5
4	6	z_6
5	7	z_7
6	7	z_7
7	7	z_7

Histogram Matching (Specification)

■ Solution:

(8) Form the histogram-specified image by mapping every every pixel whose value is s_k in the histogram-equalized image to a pixel valued z_q in the histogram-specified image as follows:

The 1120 pixels whose gray level $s_k = 0$ should be mapped to the new level $z_2 = 2$.

The 3214 pixels whose gray level was $s_k = 2$ should be mapped to $z_4 = 4$.

The 4850 pixels whose gray level was $s_k = 4$ should be mapped to $z_5 = 5$.

The 3425 pixels whose gray level was $s_k = 5$ should be mapped to $z_5 = 5$.

The 1995 pixels whose gray level was $s_k = 6$ should be mapped to $z_6 = 6$.

The 1780 pixels whose gray level was $s_k = 6$ should be mapped to $z_7 = 7$.

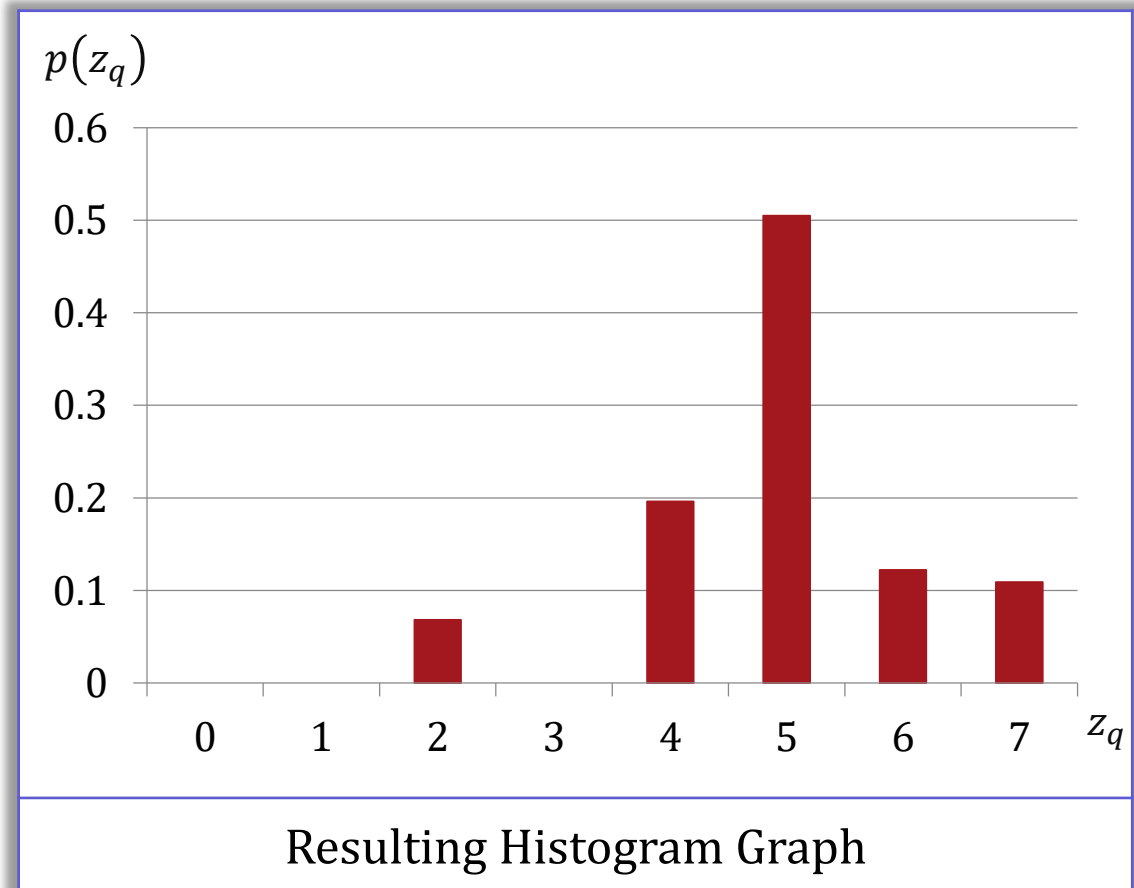
Histogram Matching (Specification)

■ Solution:

(8)

Resulting Histogram

Gray Level (z_q)	n_q	$p(z_q)$
0	0	0.000
1	0	0.000
2	1120	0.068
3	0	0.000
4	3214	0.196
5	8275	0.505
6	1995	0.122
7	1780	0.109
Total	16,384	1.000



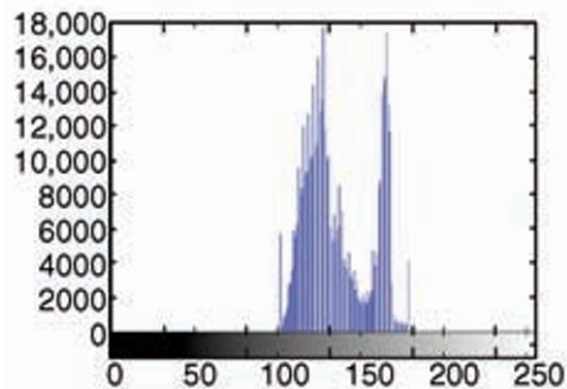
Histogram Matching (Specification)



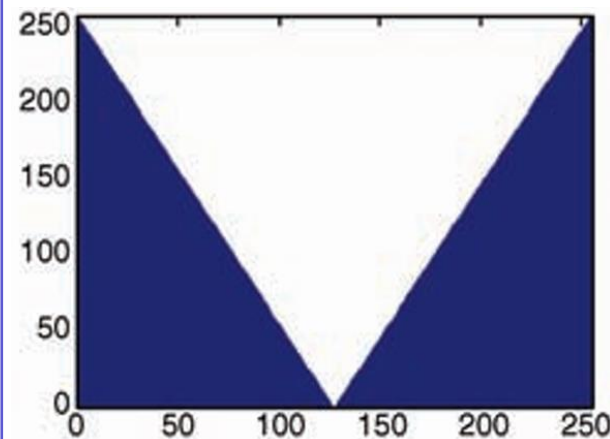
Original Image



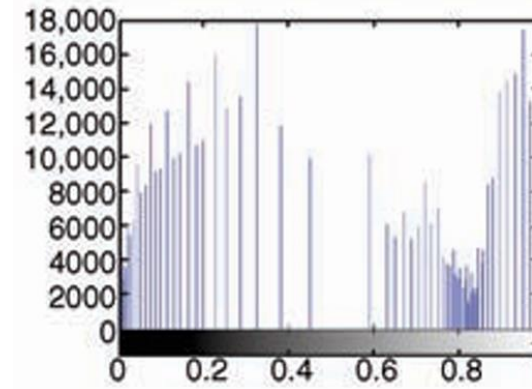
Resulting Image



Original Histogram



Desired Histogram



Resulting Histogram

Histogram Matching (Specification)– MATLAB

- The **histeq** function can also be used for histogram matching. In this case, the syntax for **histeq** usually changes to

```
Output_Img = histeq ( Input_Img , Hist)
```

where **Hist** is a 1D array of integers represents the specified (desired) histogram.

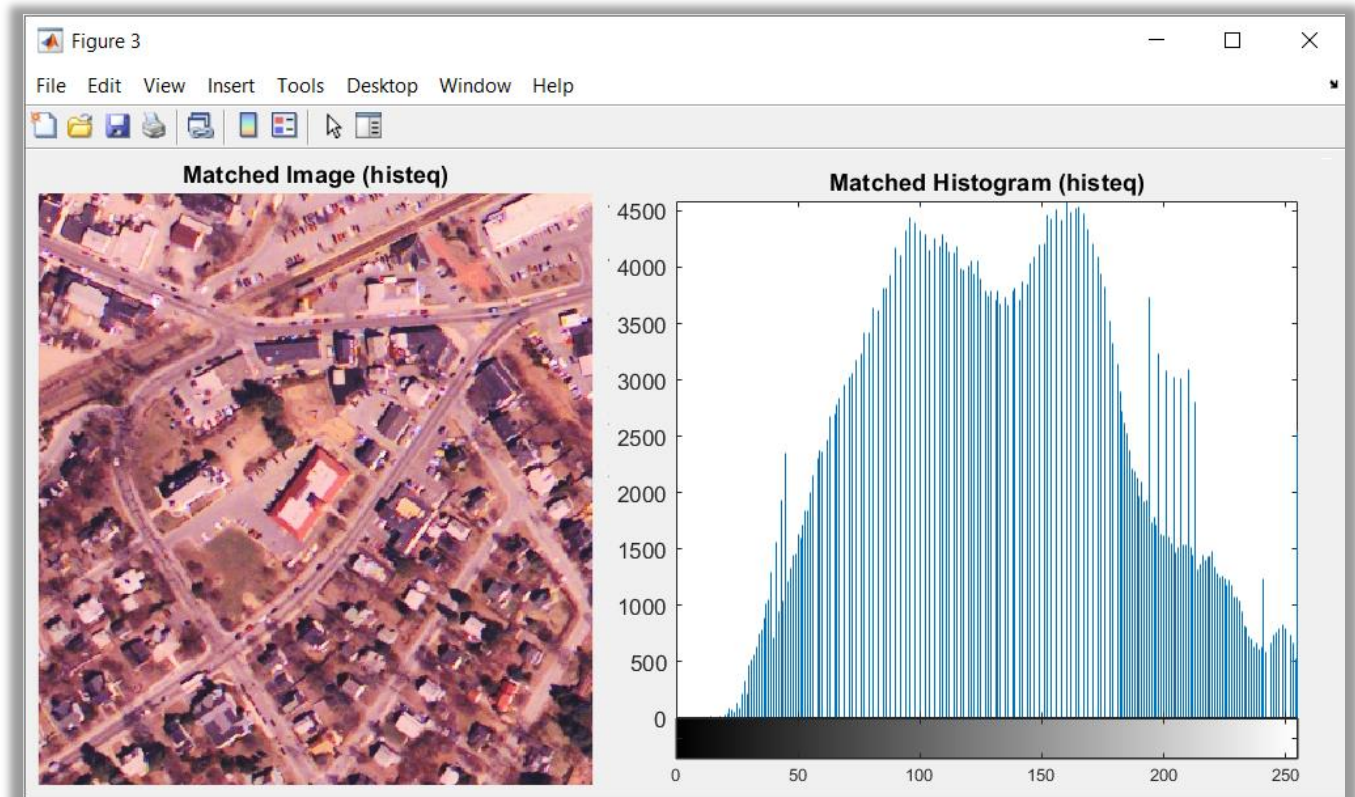
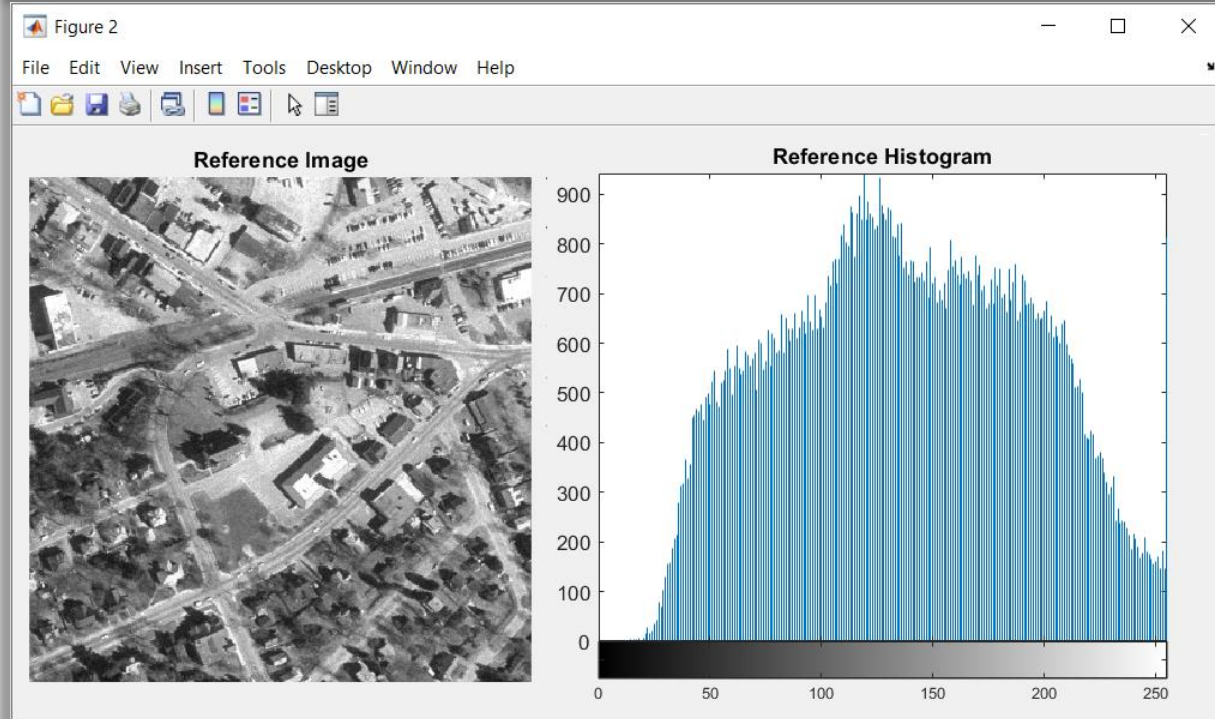
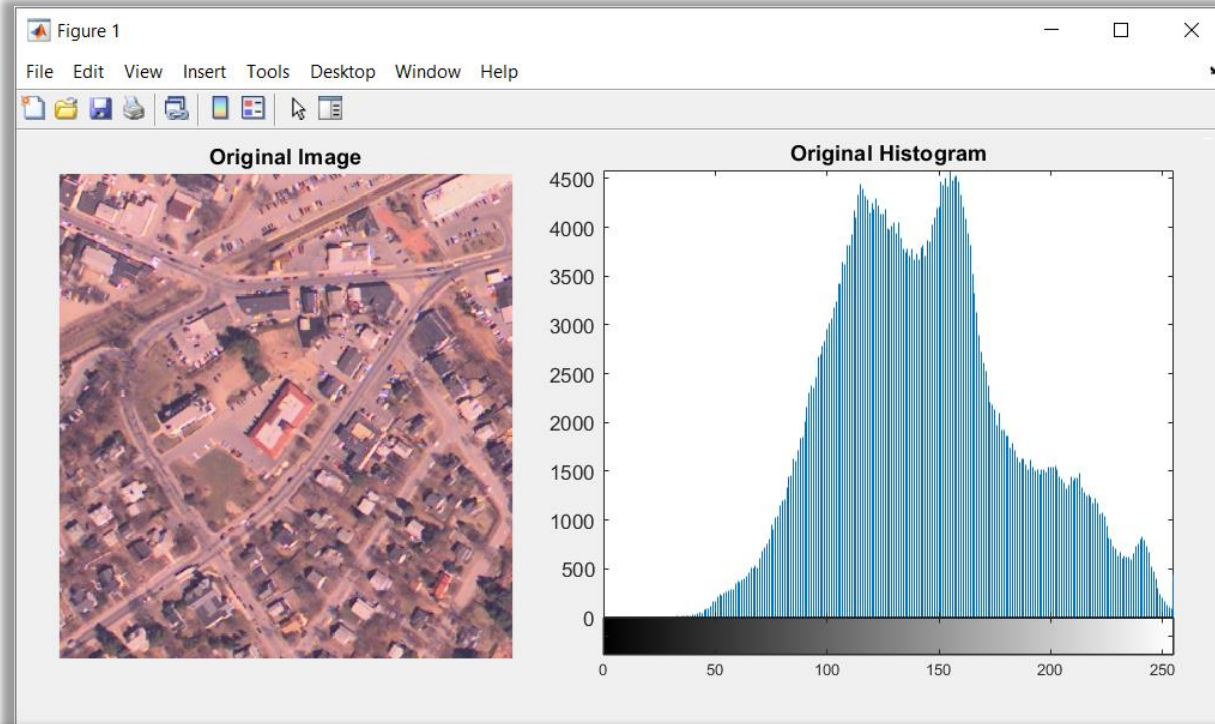
- MATLAB's IPT has also a built-in function called **imhistmatch** that can be used to adjust histogram of 2D image such that it approximately matches the histogram of another (reference) image **Ref_Img**.

```
Output_Img = imhistmatch ( Input_Img , Ref_Img )
```

Histogram Matching (Specification)– MATLAB

- **Example:** Adjust the histogram of the image “westconcordaerial.png” in MATLAB such that the the histogram approximately matches the histogram of the image “westconcordorthophoto.png”.

```
Input_Img = imread('westconcordaerial.png');  
figure  
subplot(1,2,1), imshow(Input_Img), title('Original Image')  
subplot(1,2,2), imhist(Input_Img), axis tight, title('Original Histogram')  
  
Ref_Img = imread('westconcordorthophoto.png');  
figure  
subplot(1,2,1), imshow(Ref_Img), title('Reference Image')  
subplot(1,2,2), imhist(Ref_Img), axis tight, title('Reference Histogram')  
  
Matched_Img = histeq(Input_Img, imhist(Ref_Img));  
figure  
subplot(1,2,1), imshow(Matched_Img), title('Matched Image (histeq)')  
subplot(1,2,2), imhist(Matched_Img), axis tight, title('Matched Histogram (histeq)')
```

Histogram Sliding

- This technique consists of simply adding or subtracting a constant brightness value to all pixels in the image.
- The overall effect will be an image with comparable contrast properties, but higher (for addition) or lower (for subtraction) average brightness.
- The **imadd** and **imsubtract** functions in MATLAB can be used for histogram sliding.

Histogram Sliding – MATLAB

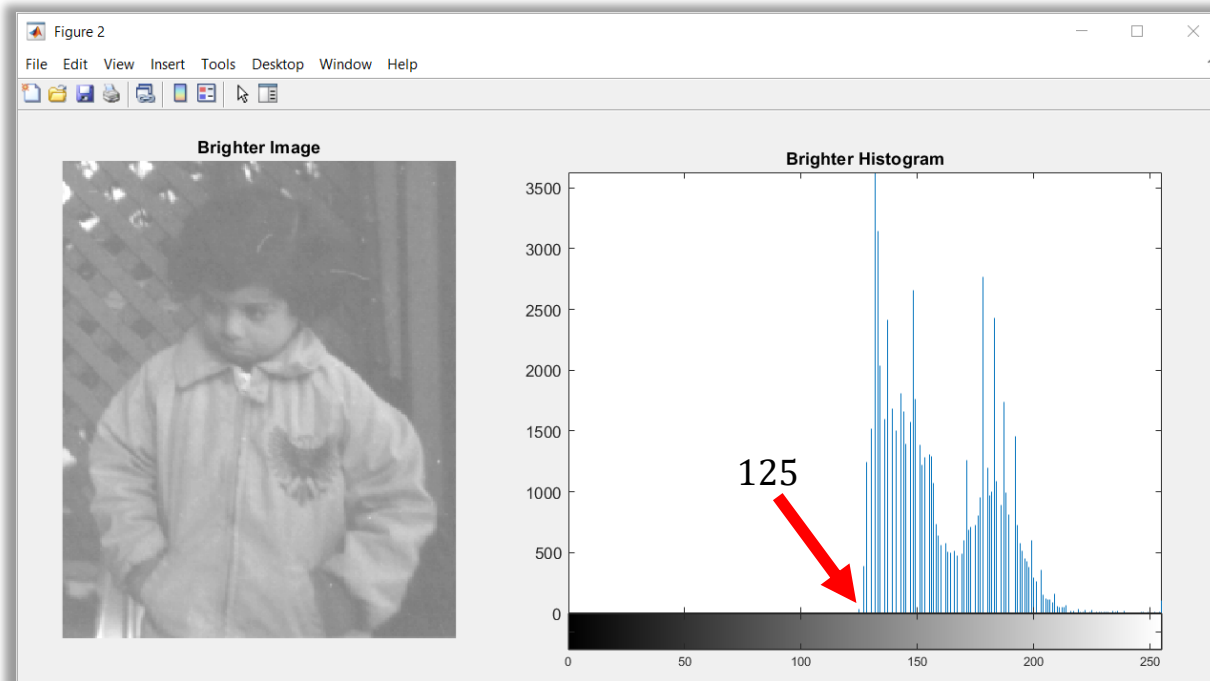
- **Example:** Perform histogram sliding by 50 to the right and left on the pout image in MATLAB.

Display the original image with its histogram and the output images with adjusted histograms.

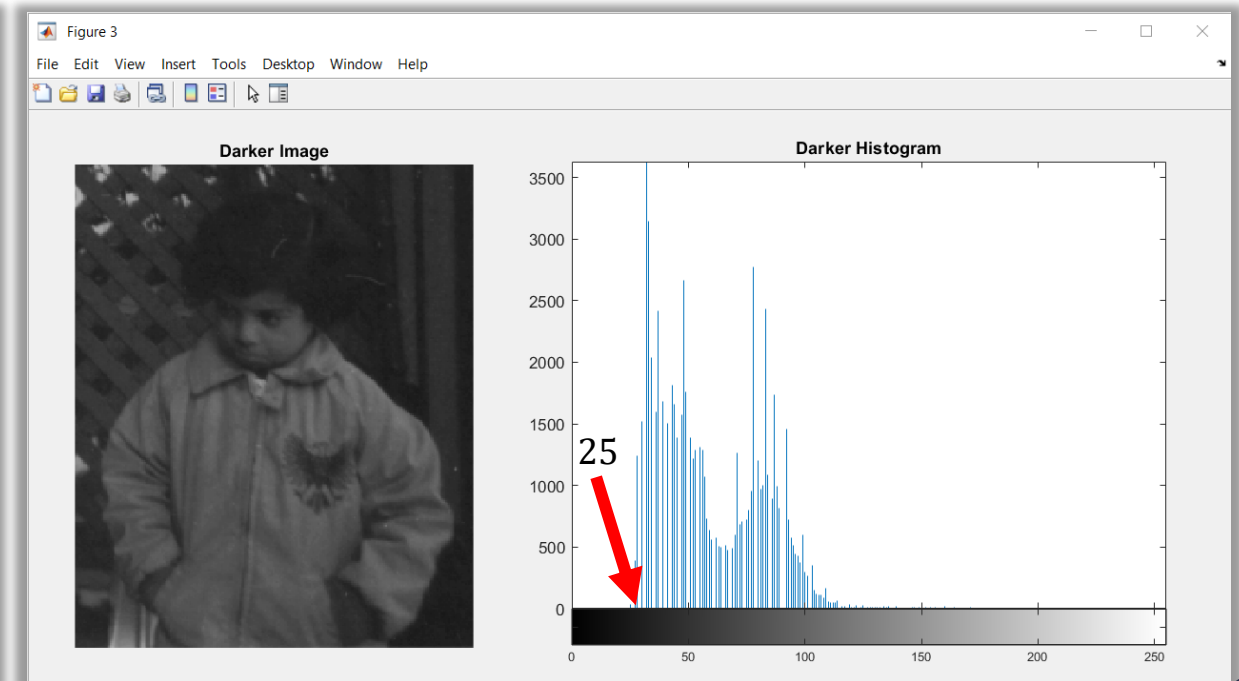
```
Input_Img = imread('pout.tif');
figure
subplot(1,2,1), imshow(Input_Img), title('Original Image');
subplot(1,2,2), imhist(Input_Img), axis tight, title('Original Histogram');

Brighter_Img = imadd(Input_Img,50);
figure
subplot(1,2,1), imshow(Brighter_Img), title('Brighter Image');
subplot(1,2,2), imhist(Brighter_Img), axis tight, title('Brighter Histogram');

Darker_Img = imsubtract(Input_Img,50);
figure
subplot(1,2,1), imshow(Darker_Img), title('Darker Image');
subplot(1,2,2), imhist(Darker_Img), axis tight, title('Darker Histogram');
```



Result of histogram sliding to the right by 50



Result of histogram sliding to the left by 50

Histogram Stretching

- Histogram stretching technique (input cropping) consists of a linear transformation that expands (stretches) part of the original histogram so that its nonzero intensity range $[r_{\min}, r_{\max}]$ occupies the full dynamic gray scale $[0, L - 1]$.
- Mathematically, each input intensity value r is mapped to an output value s according to the following linear mapping function:

$$s = \frac{r - r_{\min}}{r_{\max} - r_{\min}} \times (L - 1)$$

Histogram Stretching

- Histogram stretching increases contrast without modifying the shape of the original histogram.
- It is effective only in cases where the gray-level range of an image is compressed to a narrow range around the middle portion of the dynamic grayscale range.
- It will not be effective if applied to a poor contrast image whose histogram's lowest and highest occupied bins are close to the minimum and maximum of the full range.

Histogram Shrinking

- Histogram shrinking technique (output cropping) modifies the original histogram in such a way as to compress its dynamic grayscale range $[r_{\min}, r_{\max}]$ into a narrower gray scale, between $s_{\min}$ and $s_{\max}$.
- Mathematically, each input intensity value r is mapped to an output value s according to the following linear mapping function:

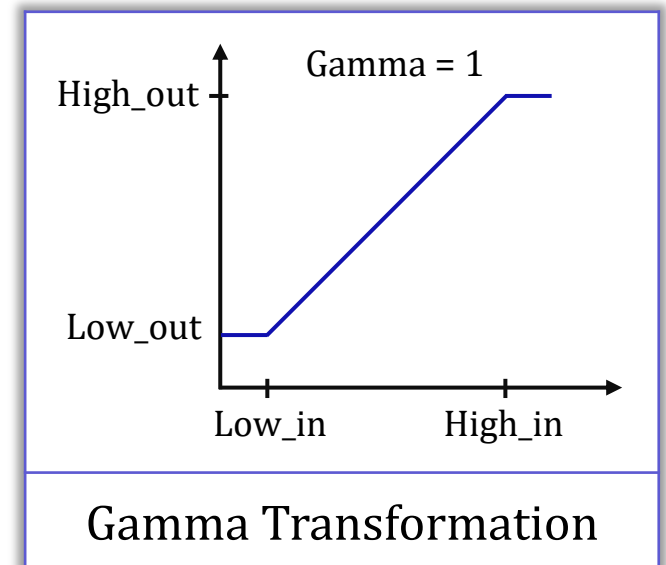
$$s = \left[\frac{s_{\max} - s_{\min}}{r_{\max} - r_{\min}} \right] (r - r_{\min}) + s_{\min}$$

Histogram Stretching and Shrinking-MATLAB

- MATLAB's IPT has also a built-in function called **imadjust** that can be used to for histogram stretching and shrinking. The syntax for the function is as follows

```
Output_Img = imadjust ( Input_Img , [ Low_in High_in ] , [ Low_out High_out ] , Gamma )
```

- Values of **Low_in**, **High_in**, **Low_out**, and **High_out** must be of type double in the range [0,1.0] and **Low_in** must be less than **High_in**.
- Any values below **Low_in** and above **High_in** are clipped or simply mapped to **Low_out** and **High_out**, respectively.
- Values in between these limits are affected by the Gamma curve for gamma equals 1 which is the default value.

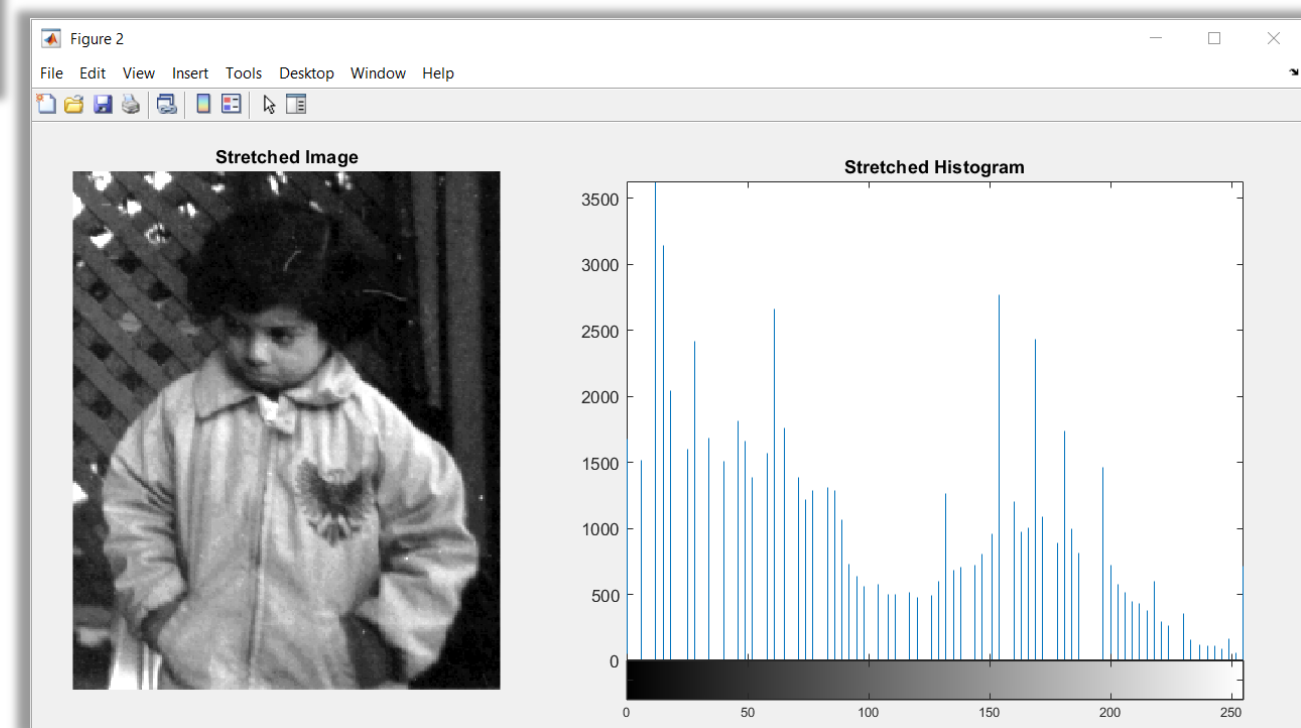


Histogram Stretching – MATLAB

- **Example:** Perform histogram stretching on the pout image in MATLAB. Display the original and output images with their histograms.

```
Input_Img = imread('pout.tif');
Stretched_Img = imadjust(Input_Img);
figure
subplot(1,2,1), imshow(Input_Img), title('Original Image')
subplot(1,2,2), imhist(Input_Img), axis tight, title('Original Histogram')
figure
subplot(1,2,1), imshow(Stretched_Img), title('Stretched Image')
subplot(1,2,2), imhist(Stretched_Img), axis tight, title('Stretched Histogram')

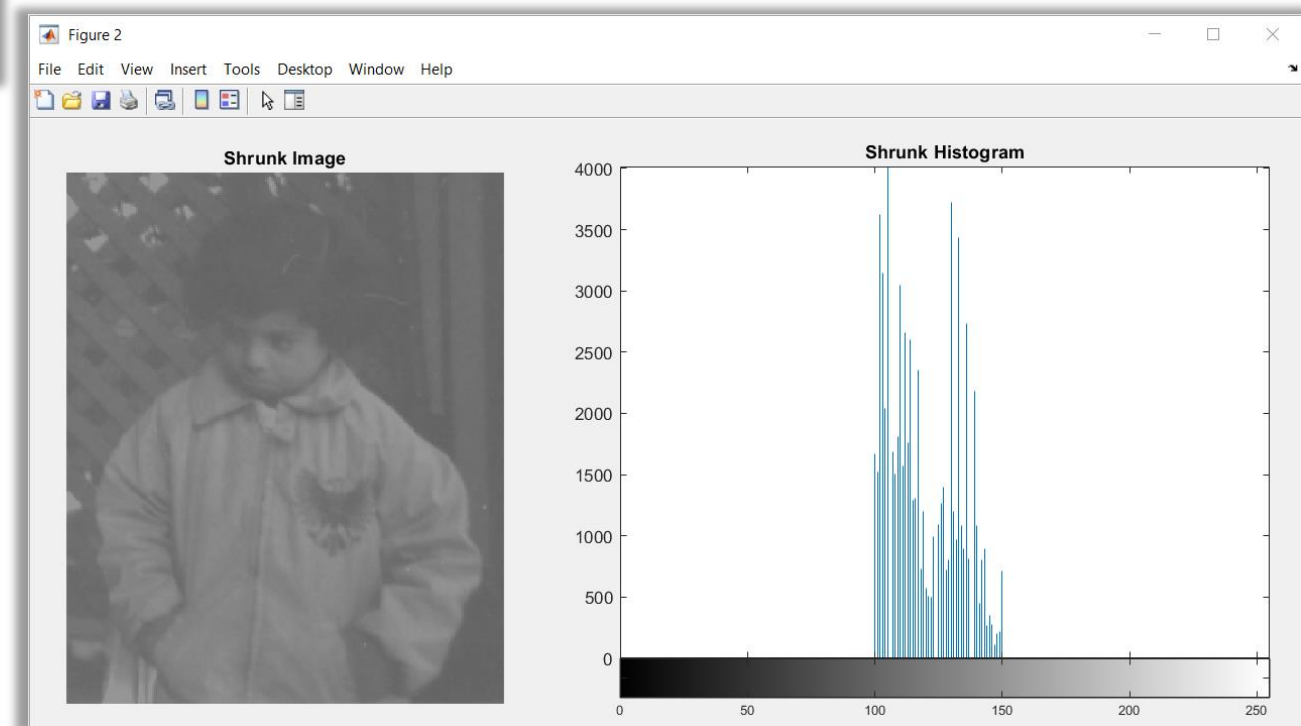
% Check the Lower and upper limits (intensity range) of the image
Img_limits = stretchlim(Input_Img);
Stretched_limits = stretchlim(Stretched_Img);
```



Histogram Shrinking – MATLAB

- **Example:** Perform histogram shrinking on the pout image in MATLAB such that the intensity range of the output image is [100,150]. Display the original and output images with their histograms.

```
Input_Img = imread('pout.tif');
Original_limits = stretchlim(Input_Img);
Desired_limits = uint8([100,150]);
Shrunk_Img = imadjust(Input_Img, Img_limits,im2double(Desired_limits));
figure
subplot(1,2,1), imshow(Input_Img), title('Original Image')
subplot(1,2,2), imhist(Input_Img), axis tight, title('Original Histogram')
figure
subplot(1,2,1), imshow(Shrunk_Img), title('Shrunk Image')
subplot(1,2,2), imhist(Shrunk_Img), axis tight, title('Shrunk Histogram')
```

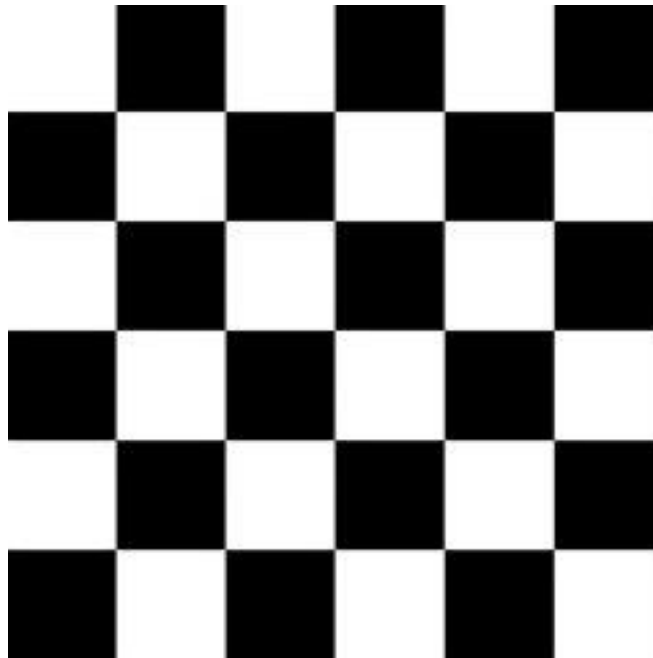


Problem

- Write a MATLAB script that implements region-based histogram equalization. Your script should allow the user to interactively select (with the mouse) a region of interest (ROI) within an image to which the histogram equalization operation will be applied.

Problem

- Write a MATLAB script that creates an 8-bit image, as shown below, with size 600×600 pixels. Then, perform the following operations separately on its histogram: **(a)** Subtraction of 130 from all pixel gray levels (histogram sliding). **(b)** Addition of 90 to all pixel gray levels. **(c)** Histogram shrinking to $[100, 150]$. Display the output image with its histogram on each case.



Thank
You